Reading the Earth from Space

A practical textbook on remote sensing, from first principles to a working project

Prepared for: Batsirai Mugomba
18 October 2026

About this book

This is a textbook for someone who writes software well and wants to become dependable at remote sensing. It teaches the physics you need, the statistics that decide whether a result is real, the engineering that makes it repeatable, and then shows how to point all of it at a new problem: methane, floods, rainfall, drought, fire, land cover. Everything runs on one worked example, the **Highveld CO_2 project** (NASA OCO-2 over South Africa's coal-power cluster), because a single concrete case teaches more than ten abstract ones.

How to read this book

Read it in order the first time. Each chapter ends with a short list of *key ideas* and a few exercises; do the exercises, because remote sensing is learned by running things and being surprised.

Every code listing in this book was **executed**, and the output printed under it is the real output from that run. Where a listing draws a figure, the figure code is left out of the page to save space, but the figure is the real result. If you retype a listing and get different numbers, you have found a difference worth understanding.

What is verified and what is not

All data in this book are **simulated** or invented for teaching, and figures say so on their face. Nothing here is a scientific finding about the real atmosphere, the real Highveld or any real facility. The project this book builds on has also not yet been run on real NASA files. Facts about missions, instruments and numbers quoted from the literature are given from general knowledge and should be checked against the mission documentation before you quote them in your own work. Where I am less sure, I say so.

The map of the book

Part	Chapters	What you learn
I. Seeing from space	1–4	What a satellite really measures, the physics of light and gas, orbits and sampling, and the data formats you will meet
II. Python as an instrument	5–7	The toolkit, raster band maths, and time-series anomalies
III. Method	8–11	Noise and uncertainty, background and detection, space and attribution, turning enhancements into emissions

Part	Chapters	What you learn
IV. Engineering	12–16	Designing a project, databases, honest graphics, testing, and delivering the work
V. Playbooks	17–19	Methane, floods/rainfall/drought/fire/land cover, and a learning plan
Appendices	A–E	Glossary, formulas, exercise answers, further reading, checklists

Conventions

- `monospace` is code, file names or variable names. **Bold** marks a term the first time it is defined.
- Boxes: blue is background, **green is a key idea**, orange is something to try, red is a warning.
- Units are written in full the first time. ppm is parts per million (10^{-6}), ppb parts per billion (10^{-9}). XCO_2 is the column-averaged dry-air mole fraction of CO_2, in ppm.
- “The project” means the Highveld CO_2 repository and its two documents (the overview and the level-by-level build tutorial).

A promise about honesty

The most valuable habit in this field is refusing to believe your own first result. You will see that habit practised on every page: the control tests, the calibration checks, the “what could be wrong” lists. A repository that says “here is what I verified and what I did not” is more impressive to an expert reader than one that claims everything works.

Chapter 1. What remote sensing really is

The one-sentence definition, and why it hides the hard part

Remote sensing is learning about something without touching it, by measuring radiation that has interacted with it. A satellite never measures CO_2, rainfall, a flood or a forest. It measures *light* (or microwave energy), counted in a detector, and everything else is inference. Whether you will be good at this work depends on how clearly you keep that fact in mind.

Compare a thermometer in a room. The thermometer's reading is almost the thing you want. A satellite's reading, a number of photons in a spectral channel, is several steps removed from what you care about, and each step adds assumptions and error. Those steps are the subject of this book.

The measurement chain

Every remote-sensing product, from a pretty false-colour picture to a national methane estimate, comes from the same chain.

Step	What happens	Where it can go wrong
1. Source	Sunlight (passive) or the instrument's own pulse (active) illuminates the scene	Sun angle, shadows, night
2. Interaction	The surface and atmosphere absorb, scatter and reflect the light	Aerosols, thin cloud, surface brightness
3. Detection	The instrument counts photons in each spectral channel	Noise, calibration drift, stray light
4. Calibration	Counts become physical radiance (Level 1)	Radiometric error, which looks like a real signal
5. Retrieval	An algorithm infers a quantity (XCO_2, reflectance, rain rate) from radiance (Level 2)	Wrong assumptions, a priori bias, algorithm artefacts
6. Gridding / compositing	Many observations are combined onto a map or time series (Level 3)	Sampling bias, mixing different conditions
7. Your analysis	You subtract a background, detect, attribute, estimate (Level 4 and beyond)	Everything in Part III of this book

Most beginners work only on step 7 and treat steps 1 to 6 as a black box that delivers “data”. Experts keep the whole chain in their heads, because a surprising result is almost always a problem at some earlier step rather than a discovery.

> **Key idea**
> A retrieved value is a **model-dependent estimate with an error budget**, not a measurement. Ask of any dataset: what was actually measured, what was assumed, and what is the stated uncertainty?

Passive and active sensing

Passive sensors measure radiation that already exists: reflected sunlight (optical sensors such as Landsat, Sentinel-2, OCO-2) or heat the Earth emits (thermal infrared). They are simple and cheap, but they need the Sun (or a warm surface) and are blocked by cloud.

Active sensors send out their own energy and time what comes back. Radar (synthetic aperture radar, SAR, such as Sentinel-1) uses microwaves, which pass through cloud and work at night. Lidar uses laser pulses and measures height and structure very precisely.

The practical consequence is simple. Floods happen under storm clouds, so optical sensors often cannot see them and radar can. Gas concentrations are measured passively from how a gas absorbs sunlight, so they need daylight and clear sky.

What is measured, and what you want

You want	The sensor measures	The bridge between them
CO_2 or CH_4 concentration	Reflected sunlight at wavelengths a gas absorbs	Radiative-transfer model plus retrieval
Vegetation health	Red and near-infrared reflectance	An empirical index such as NDVI
Open water	Optical reflectance or radar backscatter	A threshold or classifier
Rainfall	Microwave brightness, infrared cloud-top temperature, radar echoes	Calibration against rain gauges
Fire	Heat in the mid-infrared	A threshold on brightness temperature
Land cover	Spectral and textural signatures	A trained classifier

Notice the pattern. For some quantities (gases) the bridge is physics. For others (vegetation indices, classifiers) it is empirical and must be **validated** against ground truth. Knowing which kind of bridge you are crossing tells you what could be wrong.

Processing levels

NASA and ESA label products by how far they have been processed. You will meet these constantly.

Level	Meaning	Example
L0	Raw instrument data	Telemetry packets
L1	Calibrated radiance, geolocated	OCO-2 L1B spectra
L2	A retrieved geophysical variable at the instrument's own sampling	OCO-2 Lite XCO_2 soundings (what the project uses)
L3	Gridded on a regular map, usually averaged	Monthly 0.5° rainfall grids
L4	Model-assimilated or derived from many sources	Surface-flux estimates from an inversion

The project works at **Level 2**, deliberately. Level 3 averages away the individual soundings, and the individual soundings are exactly what you need to estimate noise, see plumes and count how often a place was actually observed.

Five questions to ask before touching any dataset

1. **What physical quantity does the number represent**, in what units, over what area and time?
2. **How was it retrieved**, and what does the retrieval assume?
3. **What is the sampling**: where and when does the instrument look, and where does it never look?
4. **What is the stated uncertainty**, and is it random, systematic, or both?
5. **What does the quality flag mean**, and what fraction of data did it remove?

If you cannot answer these, you do not yet know what your analysis means. Most failed remote-sensing projects fail because of questions 3 and 5, not because of any code.

> **Try it**
> Pick any dataset you have used. Write the answers to the five questions on one page. Note which answers you had to look up, and which you found you had never thought about.

Key ideas

- A satellite measures radiation; every other quantity is an inference with assumptions.
- Know the chain from source to your analysis; surprises usually come from the early links.
- Passive versus active decides what you can see through cloud, at night, and over water.
- Prefer the lowest processing level that suits the question; averaging discards information you may need.

Exercises

1. Explain in two sentences why a radar can map a flood during a storm and an optical satellite often cannot.
2. A product is called “Level 3 monthly XCO_2 at 2°”. List three things you cannot learn from it that you could learn from Level 2.
3. For each of NDVI, rainfall and XCO_2, say whether the bridge from measurement to quantity is physical or empirical, and what that implies for validation.

Chapter 2. Light, matter and the atmosphere

The electromagnetic spectrum as a toolbox

Light is an electromagnetic wave. What differs between X-rays, visible light and radio is only wavelength (or, equivalently, frequency and energy). For remote sensing, think of wavelength regions as different *tools*, each good at certain things.

Region	Wavelength	Typical use
Visible	0.4–0.7 µm	Colour, true-colour images, chlorophyll absorbs red and blue
Near-infrared (NIR)	0.7–1.3 µm	Healthy leaves reflect strongly; water absorbs; the O_2 A-band at 0.76 µm
Shortwave infrared (SWIR)	1.3–3 µm	Water content, burn scars, **CO_2 (1.6 and 2.06 µm) and CH_4 (1.65 and 2.3 µm)**
Thermal infrared	3–14 µm	Surface and cloud temperature, fire, some gases
Microwave	1 mm–1 m	Radar, rain, soil moisture; passes through cloud

Two facts organise almost everything. First, **different materials reflect and absorb differently by wavelength**, which is why a spectrum can identify them. Second, **the atmosphere is transparent in some wavelength "windows" and opaque in others**. We build instruments to look through windows (to see the surface) or into the absorption bands (to see the gases).

Spectral signatures of surfaces

A surface's **reflectance** is the fraction of incoming light it sends back, as a function of wavelength. A few shapes are worth memorising because they power most of the indices in this book.

- **Healthy vegetation** is dark in red (chlorophyll absorbs it) and very bright in near-infrared (leaf structure scatters it). The jump between red and NIR is called the *red edge*. This is the basis of NDVI.
- **Water** is dark everywhere and nearly black in NIR and SWIR. That makes water stand out against soil and plants in those bands, the basis of NDWI and MNDWI (Chapter 6).
- **Bare soil** reflects moderately and rises smoothly with wavelength.
- **Burnt ground** is dark in NIR and brighter in SWIR, the opposite of healthy vegetation, the basis of burn-severity indices.
- **Shadows and dark surfaces** are dark in every band. They can masquerade as water, because water is also dark. A great deal of practical remote sensing is arguing with shadows.

Why gases leave fingerprints

A molecule absorbs light only at specific wavelengths, where the photon energy matches a transition between its vibration and rotation states. Each gas therefore has a pattern of narrow absorption **lines**, its fingerprint. CO_2, CH_4, H_2O and O_2 all have rich line structure in the shortwave infrared. At ordinary atmospheric pressure each line is broadened into a smooth bump by molecular collisions, and its depth depends on how many absorbers the light passed through.

Beer–Lambert: the law behind every gas measurement

If light of intensity I_0 passes through an absorbing layer, the intensity that emerges is

```
I = I0 * exp(-tau)        tau = sum over absorbers of (cross-section x number per area)
```

where **tau** is the *optical depth*. Double the amount of absorber and tau doubles, so the transmitted fraction falls exponentially. For sunlight reflected from the ground to a satellite the light crosses the atmosphere twice, down and up, so the path is longer than the vertical column by a factor called the **air mass**, roughly 1/cos(solar zenith angle) + 1/cos(viewing angle).

Here is a toy model of one band, with invented lines. It is not real spectroscopy, but it has the right structure, and the numbers it prints teach the main lesson.

```
"""s01 - Why a satellite can 'see' CO2: Beer-Lambert absorption in a toy band."""
import numpy as np

def lorentz(nu, nu0, gamma):
    """Pressure-broadened line shape (unit area)."""
    return (gamma / np.pi) / ((nu - nu0) ** 2 + gamma ** 2)

nu = np.linspace(6200.0, 6240.0, 4000)              # wavenumber, cm-1 (about 1.61 micrometres)
rng = np.random.default_rng(1)
centres = np.sort(rng.uniform(6202, 6238, 14))       # 14 invented CO2 lines
strength = rng.uniform(0.2, 1.0, 14)

def optical_depth(co2_scale):
    tau = np.zeros_like(nu)
    for c, s in zip(centres, strength):
        tau += co2_scale * 0.35 * s * lorentz(nu, c, 0.08)
    return tau

def radiance(co2_scale, sza=30.0, vza=0.0):
    """I / I0 after the sunlight goes down AND back up through the atmosphere."""
    airmass = 1 / np.cos(np.radians(sza)) + 1 / np.cos(np.radians(vza))
    return np.exp(-airmass * optical_depth(co2_scale))

ref = radiance(1.0)
more = radiance(1.0 + 4.0 / 410.0)                   # +4 ppm on a 410 ppm background
change = 100 * (more.mean() / ref.mean() - 1)
print(f"deepest line absorbs {100 * (1 - ref.min()):.0f}% of the light")
print(f"band-mean transmittance: {ref.mean():.4f}")
print(f"+4 ppm CO2 (about +1%) changes band-mean radiance by {change:.3f}%")
print(f"=> 1 ppm needs radiometric precision of about {abs(change) / 4:.3f}%")
```

Output

```
deepest line absorbs 95% of the light
band-mean transmittance: 0.8995
+4 ppm CO2 (about +1%) changes band-mean radiance by -0.073%
=> 1 ppm needs radiometric precision of about 0.018%
```

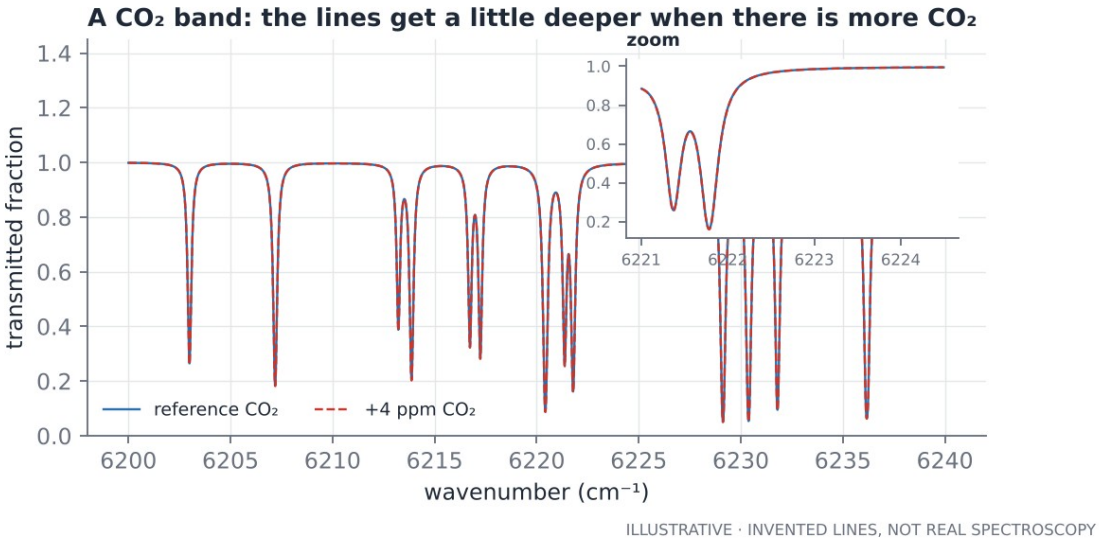


Figure 2.1. A toy CO_2 band. The extra 4 ppm makes the lines only slightly deeper, which is why measuring ppm demands such precise radiometry.

Read the output carefully. The deepest line absorbs 95 % of the light, yet adding 1 % more CO_2 changes the band-mean radiance by less than 0.1 %. Two reasons: the strongest lines are nearly *saturated* (absorbing almost everything already), so extra gas barely changes them; and most of the band is not on a line at all. The signal that carries the information is a small difference between large numbers. That is why CO_2 satellites need very accurate calibration and why their retrievals are statistical, not simple.

> **Real instrument requirements**
>
> Published OCO-2 papers state the required column accuracy as a fraction of a percent, about 1 ppm on 400 ppm, with a target of about 0.25 %. The toy above is not a substitute for those numbers; look them up in the mission papers when you need them.

Using several bands at once

A single CO_2 absorption band is not enough. The light path through the atmosphere is unknown because of clouds, aerosols and surface pressure. OCO-2 therefore measures three bands: the **O_2 A-band** (0.76 µm), which provides the light path because O_2 is well mixed at a known fraction of air; a **weak CO_2 band** (1.61 µm); and a **strong CO_2 band** (2.06 µm). Dividing the CO_2 absorption by the O_2 absorption cancels much of the unknown path and leaves a mixing ratio. For methane, TROPOMI uses a band near 2.3 µm together with a near-infrared band for the path.

The general principle is worth remembering as a design pattern: **measure the thing you want and a reference that shares the same unknown, then take the ratio.**

Column-averaged mole fractions

A satellite looks down through the whole atmosphere, so it measures a *column*, not a concentration at a height. The quantity reported is the **column-averaged dry-air mole fraction**:

```
XCO2 = (number of CO2 molecules above a patch of ground) / (number of dry-air molecules above it)
     = sum over layers of  [ pressure weight x CO2 mole fraction ]
```

The pressure weight of a layer is its share of the total column of air, which in a hydrostatic atmosphere is simply its pressure thickness divided by surface pressure. Dividing by dry air matters: it makes XCO_2

insensitive to surface pressure and water vapour, so a plateau or a storm does not masquerade as a change in CO_2.

```
"""s02 - What 'column-averaged' means, and why a 20 ppm surface plume is only ~1 ppm in XCO2."""
import numpy as np

G, M_DRY, M_CO2 = 9.80665, 0.0289652, 0.04401        # m/s2, kg/mol, kg/mol

def column_dry_air(ps_hpa):
    """Moles of dry air above each m2 of ground (hydrostatic balance)."""
    return ps_hpa * 100.0 / (G * M_DRY)

def profile(p_hpa):
    """Invented CO2 profile: 410 ppm near the ground, falling to 402 ppm at the top."""
    return 410.0 - 8.0 * (1 - p_hpa / 1013.0) ** 2

def xco2(ps_hpa, plume_ppm=0.0, plume_depth_hpa=50.0, n=2000):
    """Pressure-weighted average of the mole fraction over the whole column."""
    edges = np.linspace(0.0, ps_hpa, n + 1)
    mid, dp = 0.5 * (edges[1:] + edges[:-1]), np.diff(edges)
    x = profile(mid) + np.where(mid > ps_hpa - plume_depth_hpa, plume_ppm, 0.0)
    return np.sum(x * dp) / ps_hpa

for ps, place in ((1013.0, "sea level"), (850.0, "Highveld, ~1500 m")):
    n_air = column_dry_air(ps)
    print(f"{place:>18}: ps={ps:6.1f} hPa  dry air={n_air:8.0f} mol/m2  "
          f"CO2 column={n_air * xco2(ps) * 1e-6:6.3f} mol/m2  XCO2={xco2(ps):7.3f} ppm")

bump = xco2(850.0, plume_ppm=20.0) - xco2(850.0)
print(f"\n+20 ppm in the lowest 50 hPa (~500 m) raises XCO2 by {bump:.2f} ppm "
      f"(= 20 x 50/850 = {20 * 50 / 850:.2f})")
kg_per_ppm = column_dry_air(850.0) * 1e-6 * M_CO2
print(f"1 ppm of XCO2 at 850 hPa = {kg_per_ppm * 1000:.2f} g of CO2 per m2 of ground")
```

Output

```
        sea level: ps=1013.0 hPa  dry air=  356625 mol/m2  CO2 column=145.265 mol/m2  XCO2=407.333 ppm
 Highveld, ~1500 m: ps= 850.0 hPa  dry air=  299241 mol/m2  CO2 column=121.742 mol/m2  XCO2=406.835 ppm

+20 ppm in the lowest 50 hPa (~500 m) raises XCO2 by 1.18 ppm (= 20 x 50/850 = 1.18)
1 ppm of XCO2 at 850 hPa = 13.17 g of CO2 per m2 of ground
```

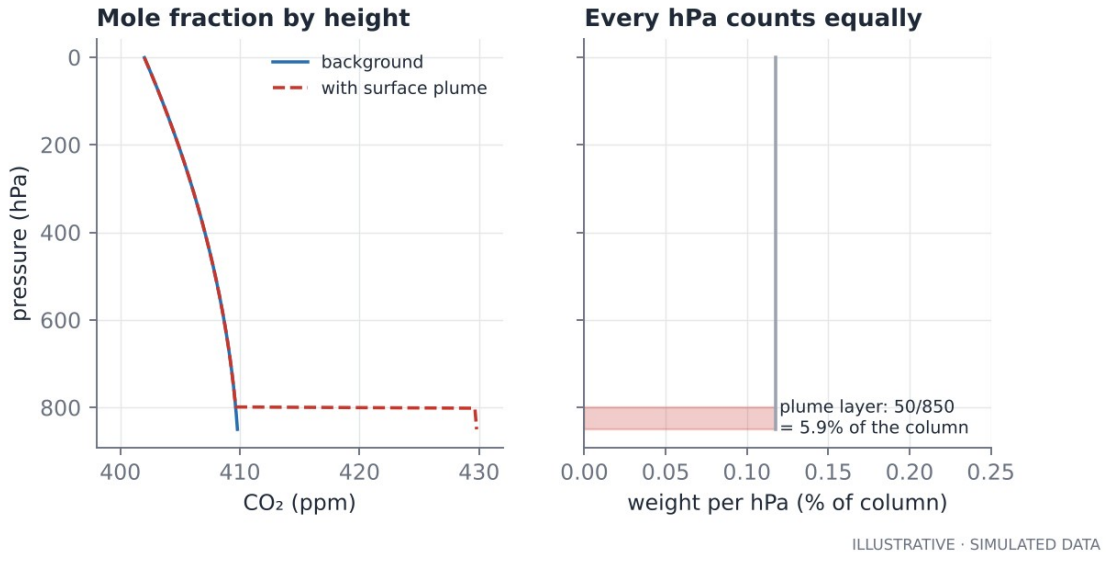


Figure 2.2. A 20 ppm plume in the lowest 50 hPa is about 6 % of the column, so it adds only about 1.2 ppm to XCO_2.

This listing carries two lessons you will use repeatedly.

1. **A big surface signal is a small column signal.** A 20 ppm rise in the bottom 500 m, which a ground station would call dramatic, moves XCO_2 by about 1 ppm. A column measurement dilutes local signals by the ratio of the plume depth to the whole atmosphere. This is why enhancement thresholds in the project are near 1 ppm.
2. **The unit conversion is worth knowing by heart.** At the Highveld's surface pressure (about 850 hPa, because the plateau is high), 1 ppm of XCO_2 equals roughly 13 g of CO_2 per square metre of ground. Every emission estimate in Chapter 11 starts from that number.

Vertical sensitivity and the averaging kernel

The satellite is not equally sensitive at all heights. Its **averaging kernel** says how much a change at each height shows up in the retrieved column. OCO-2's kernel is close to 1 throughout the troposphere, which is why it sees surface sources well, but retrievals also lean towards an **a priori** profile. For a point-source plume this matters little, because you subtract a nearby background. For comparing satellite columns with models or aircraft profiles it matters a lot, and you must apply the kernel to the model before comparing. If you ever compare a satellite number with a model number and skip this, you are comparing two different quantities.

What disturbs the light

- **Aerosols and thin cirrus** scatter light and change the path. Retrievals try to model them; leftover errors appear as spurious XCO_2 changes correlated over tens of kilometres.
- **Surface brightness.** Very dark surfaces (water, burn scars, some lava fields) give little light and noisy retrievals; very bright surfaces (salt pans, snow) can saturate or bias them. This is why XCO_2 products often exclude water outside glint mode.
- **Topography.** Surface pressure and path length change with elevation. High, rough terrain is a recognised source of retrieval artefacts.
- **Cloud** simply blocks the view. It is usually removed by the quality flag, which is why far fewer soundings survive than are collected.

> **The central warning of this book**
>
> Retrieval artefacts look exactly like small plumes: a localised, coherent enhancement of about 1 ppm that is not a source. You cannot recognise them by eye. You can only measure how often your method flags them, using **control regions** with no sources. Chapter 8 and Chapter 10 build that test.

Key ideas

- A gas has a line-structured absorption fingerprint; Beer–Lambert makes the transmitted light fall exponentially with amount.
- CO_2 changes the signal only slightly and partially saturates, so the signal is a small difference between large numbers.
- Use a reference gas (O_2) to cancel the unknown light path: measure the target and a shared-unknown reference, then take the ratio.
- XCO_2 is a pressure-weighted column average; a 20 ppm surface plume is about 1 ppm in the column.
- Artefacts mimic plumes; controls, not eyeballs, separate them.

Exercises

1. Using `s02`, change the plume depth to 100 hPa and the surface pressure to 1013 hPa. By how much does XCO_2 change for a 20 ppm plume? Predict first, then run it.
2. In `s01`, double the line widths. What happens to the saturation of the deepest line, and to the change in band-mean radiance for +4 ppm? Why?
3. Explain in your own words why dividing by dry air, not total air, makes XCO_2 robust to weather.

Chapter 3. Platforms, orbits and sampling

Why geometry decides what science is possible

Before you open a single file, ask: *where and when did this instrument look, and where did it never look?* A beautiful algorithm cannot detect a plume the satellite did not fly over. Sampling is the most underrated part of remote sensing, and it determines which questions a dataset can answer at all.

Orbits in one page

- **Low Earth orbit (LEO)**, 400–1,000 km up, circles the Earth in about 90–100 minutes. Almost all Earth-observation satellites are here, because they are close enough for fine detail.
- **Sun-synchronous orbit (SSO)** is a near-polar LEO tuned so the satellite crosses every latitude at the same *local solar time* each day. Constant illumination makes images comparable over years. Landsat, Sentinel-2, OCO-2 and TROPOMI all use it (with different crossing times: OCO-2 and TROPOMI near early afternoon, when the Sun is high and the boundary layer is well mixed).
- **Geostationary orbit (GEO)**, 35,786 km up, matches Earth's rotation, so the satellite hovers over one spot and sees a whole hemisphere every few minutes, but with coarse detail. Weather satellites live here.
- **The International Space Station** is a low-inclination LEO (about 52°), not sun-synchronous. Instruments there (OCO-3, EMIT) see a changing local time and only the latitudes between about ±52°.

The ground track and the repeat cycle

An orbit traces a line on the ground, the **ground track**. Earth rotates beneath it, so successive orbits fall further west. Orbits are usually designed to repeat: after a fixed number of orbits in a fixed number of days, the track retraces itself exactly. OCO-2's pattern is 233 orbits in 16 days.

The arithmetic is worth doing yourself, because it produces the numbers that govern the whole project.

```
"""s03 - Orbit arithmetic: why a 10 km swath crosses a 4 degree box only 2-3 times per cycle."""
import numpy as np

ORBITS, DAYS = 233, 16                    # OCO-2 repeats its ground track every 233 orbits = 16 days
EARTH_CIRC_KM, SWATH_KM = 40075.0, 10.3

period_min = DAYS * 1440 / ORBITS
orbits_per_day = ORBITS / DAYS
spacing_equator = EARTH_CIRC_KM / ORBITS              # gap between neighbouring tracks, whole cycle
print(f"orbital period            : {period_min:.1f} min  ({orbits_per_day:.2f} orbits/day)")
print(f"track spacing at equator  : {spacing_equator:.0f} km")

lat, box_width_deg = -26.0, 4.0
spacing_at_lat = spacing_equator * np.cos(np.radians(lat))
box_width_km = box_width_deg * 111.32 * np.cos(np.radians(lat))
print(f"track spacing at {lat:.0f} deg  : {spacing_at_lat:.0f} km (E-W)")
print(f"box width at {lat:.0f} deg      : {box_width_km:.0f} km")
print(f"tracks crossing the box   : {box_width_km / spacing_at_lat:.1f} per 16-day cycle")
print(f"swath fraction of ground  : {100 * SWATH_KM / spacing_at_lat:.1f}% "
      f"(only this share of the box is ever measured)")
print(f"cycles per year           : {365.25 / DAYS:.1f}  ->  ~{365.25 / DAYS * box_width_km /
spacing_at_lat:.0f} "
      f"passes/yr before clouds")
```

Output

```
orbital period            : 98.9 min  (14.56 orbits/day)
```

```
track spacing at equator  : 172 km
track spacing at -26 deg  : 155 km (E-W)
box width at -26 deg      : 400 km
tracks crossing the box   : 2.6 per 16-day cycle
swath fraction of ground  : 6.7% (only this share of the box is ever measured)
cycles per year          : 22.8  ->  ~59 passes/yr before clouds
```

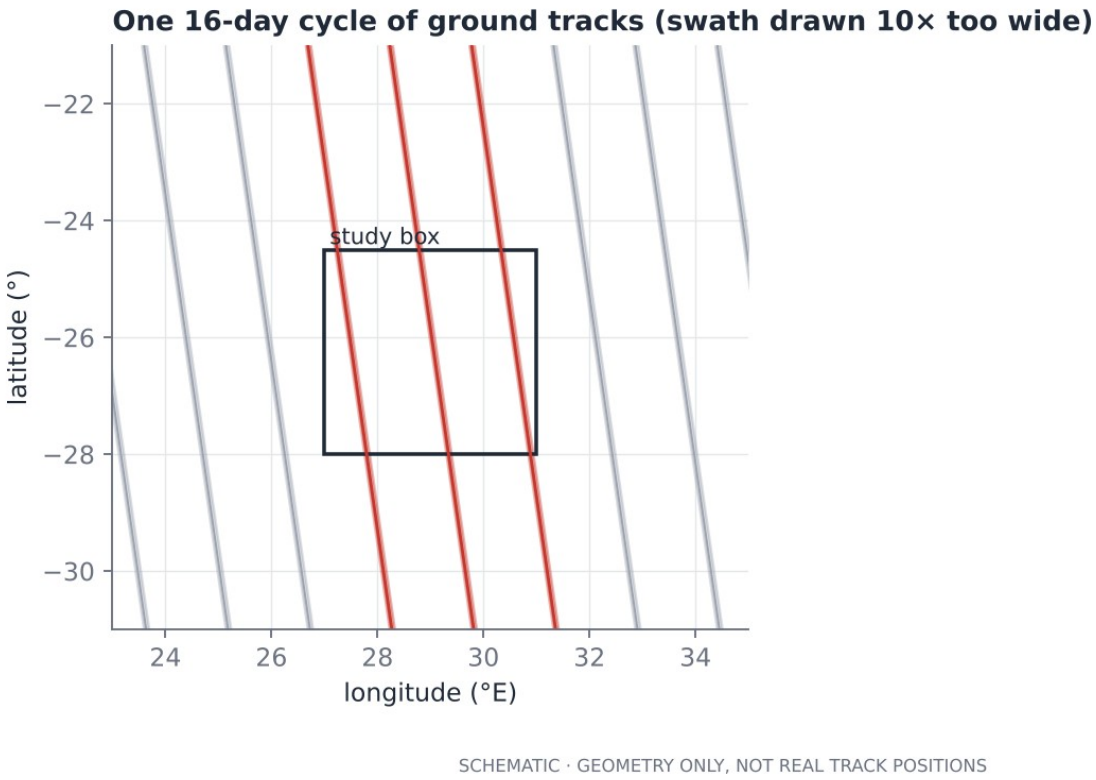


Figure 3.1. Ground tracks of one 16-day cycle across the study box, with the swath drawn ten times too wide so you can see it. Tracks that cross the box are red.

The listing says that over one 16-day cycle the tracks are spaced about 155 km apart east–west at 26°S, the box is 400 km wide, so about 2.6 tracks cross it, and each track measures a swath only about 10 km wide, **less than 7 % of the ground**. Before clouds, that is about 59 passes per year, and (after the cloud fraction assumed in Chapter 10) a few dozen usable ones. These are not details; they are the reason a “detection” here is rare, a non-detection is weak evidence, and the honest summary statistic is a detection *frequency* with an interval.

The four resolutions

Satellite data have four kinds of resolution, and they trade against each other.

Resolution	Meaning	Example values
Spatial	Size of one pixel or footprint	OCO-2 about 1.3 × 2.3 km; TROPOMI about 5.5 × 3.5 km; Sentinel-2 10 m; Landsat 30 m
Spectral	Number and width of wavelength bands	Landsat: 7–8 bands; Sentinel-2: 13; imaging spectrometers: hundreds of narrow bands
Temporal	How often the same place is observed	Geostationary: minutes; Sentinel-2: about 5 days; OCO-2: 16-day repeat track

Resolution	Meaning	Example values
Radiometric	How finely brightness is quantised, and how noisy it is	12–16 bit; signal-to-noise ratio is what matters in practice

You cannot maximise all four. A wide swath with fine pixels needs enormous data rates. A fine spectrum over a wide area needs a big telescope. The instrument builders pick the combination that serves one science goal, and you inherit their choice.

Imaging versus sounding

Imagers form pictures (Sentinel-2, Landsat): contiguous pixels over a wide swath, good for mapping extent. **Sounders** such as OCO-2 take a limited number of narrow footprints with high spectral precision, good for measuring a quantity accurately but leaving gaps between tracks. TROPOMI sits between the two, with a wide swath of kilometre-scale pixels and good spectral resolution, giving daily global coverage for a few trace gases. Remember this when a method designed for images (clustering pixels, flood mapping) meets sounding data (sparse samples along tracks).

A map of missions worth knowing

This table is for orientation, not citation. Details change (launch dates, constellation members, retired satellites); confirm against the mission page before you rely on any number.

Mission or product	What it provides	Why you might use it
OCO-2, OCO-3 (NASA)	Column CO_2 (also solar-induced fluorescence); OCO-3 on the ISS	Point-source and regional CO_2; the project's data
GOSAT, GOSAT-2 (Japan)	Column CO_2 and CH_4	Long CH_4 and CO_2 records, sparse sampling
TROPOMI on Sentinel-5P (ESA)	Daily global CH_4, NO_2, CO and other gases at kilometre scale	Methane hotspots and large emitters; NO_2 for combustion
Sentinel-2 (ESA)	13-band optical imagery, 10–60 m, revisit about 5 days	Floods, vegetation, burn scars; some methane detections at very large sources
Landsat 8 and 9 (USGS/NASA)	30 m optical and thermal, long archive back to the 1980s	Change over decades
Sentinel-1 (ESA)	C-band radar, cloud-penetrating	Flood extent, deformation
MODIS, VIIRS	Daily global imagery, fire detections, vegetation indices	Active fire, large-scale vegetation
GPM IMERG (NASA/JAXA)	Merged global rainfall at about 0.1°, 30-minute steps	Rainfall, flood forcing
EMIT (NASA, on the ISS)	Imaging spectrometer, tens-of-metres pixels	Targeted methane plume imaging
GHGSat and similar commercial missions	Tens-of-metres methane imagery of selected sites	Facility-scale methane

Consequences of sampling you must plan for

- **Sparse sampling means high variance.** Few observations of a place produce wide confidence intervals on any frequency you estimate (Chapter 10).
- **Sampling is not random.** The satellite always crosses at the same local time, so everything it sees is conditioned on early-afternoon conditions: boundary layer deep, winds moderate, emissions at their daily pattern. A "mean" from it is a *mean at 13:30 on clear days*, not a daily or annual mean.
- **Clear-sky bias.** Cloud removes soundings, and cloudy days are meteorologically different (often wet season, different winds). Your sample is biased towards fair weather.
- **Coverage must be reported, not hidden.** Always record, for each target, how many times it was actually observed. A non-detection with ten good looks and a non-detection with one look are different findings.

> **Key idea**
> The sampling pattern is part of the data. Report observations alongside detections, and never describe a sparse, clear-sky, fixed-local-time sample as if it were a full record.

Key ideas

- Geometry (orbit, swath, repeat cycle, local time) sets what is observable before any algorithm runs.
- Spatial, spectral, temporal and radiometric resolution trade against each other.
- Imagers map extent; sounders measure a quantity precisely but sparsely.
- Report coverage and observation counts as first-class results.

Exercises

1. Using the structure of `s03`, compute how many tracks of a mission with a 233-orbit/16-day pattern would cross a 1°-wide box at the equator, and what fraction of ground a 10 km swath covers there.
2. A friend says, "OCO-2 saw no plume at that plant, so it was not emitting." List four reasons that conclusion is unsafe.
3. Choose a sensor from the table for (a) a flood during a storm, (b) a methane leak from one well pad, (c) a decade of vegetation change. Justify each by resolution trade-offs.

Chapter 4. Data: formats, coordinates and access

Files you will meet

Satellite data come in a handful of container formats. Knowing what is inside each saves hours.

Format	What it is	Typical use
NetCDF4 / HDF5	Self-describing containers of named, multi-dimensional arrays in groups, with attributes for units and fill values	Atmospheric products, climate data, OCO-2, TROPOMI
GeoTIFF / Cloud-Optimised GeoTIFF	An image with a coordinate system, optionally tiled so a small window can be read over the network	Optical imagery, rasters, DEMs
Zarr	Chunked arrays stored as many small files, built for cloud storage and parallel reads	Large time-series cubes
Parquet / CSV	Tabular data	Soundings after you have extracted them; results
GeoJSON / GeoPackage / Shapefile	Vector geometry (points, lines, polygons) with attributes	Facility locations, boundaries, event footprints

The rule of thumb: **arrays stay in NetCDF/GeoTIFF/Zarr, tables go to Parquet or a database, geometry goes to GeoPackage/GeoJSON or PostGIS.** The project follows it: NetCDF in, CSV and PostGIS out.

Reading a NetCDF file

NetCDF files are organised in **groups** (like folders), containing **variables** (arrays) with **dimensions** and **attributes**. OCO-2 Lite files hold one long dimension called `sounding_id`, with root variables such as `xco2`, `latitude` and `time`, and extra variables in groups (`Sounding/operation_mode`). The first thing to do with any new file is look inside it, which is why the project has an `inspect_file` tool.

The listing below builds a small file shaped like that, then reads it back with xarray, showing the three habits that save you most often: handle fill values, subset with a mask, and hand over to pandas.

```
"""s04 - Reading a NetCDF file the way OCO-2 Lite files are laid out (one long 'sounding' dimension)."""
import numpy as np
import xarray as xr

FILL = -999999.0
rng = np.random.default_rng(0)
n = 2000
lat = rng.uniform(-30, -22, n).astype("float32")
lon = rng.uniform(25, 33, n).astype("float32")
xco2 = 410 + rng.normal(0, 0.8, n)
xco2[rng.random(n) < 0.03] = FILL                       # a few 'no retrieval' values
t = np.datetime64("2023-06-01") + (np.sort(rng.uniform(0, 86400, n)) * 1e9).astype("timedelta64[ns]")

ds = xr.Dataset({
    "xco2": ("sounding", xco2.astype("float32")),
    "xco2_quality_flag": ("sounding", (rng.random(n) < 0.35).astype("int8")),
    "latitude": ("sounding", lat, {"units": "degrees_north"}),
    "longitude": ("sounding", lon, {"units": "degrees_east"}),
    "time": ("sounding", t),
})
```

```
ds.to_netcdf("/home/claude/book/work/demo.nc", encoding={"xco2": {"_FillValue": FILL}})

raw = xr.open_dataset("/home/claude/book/work/demo.nc", mask_and_scale=False)
print("without masking, fill values hide in the data:", int((raw.xco2 == FILL).sum()),
      "| mean =", round(float(raw.xco2.mean()), 1), "ppm  <- nonsense")

ds = xr.open_dataset("/home/claude/book/work/demo.nc")           # masking is the default
print("with masking, they become NaN:", int(ds.xco2.isnull().sum()),
      "| mean =", round(float(ds.xco2.mean()), 3), "ppm")

box = (ds.longitude >= 27) & (ds.longitude <= 31) & (ds.latitude >= -28) & (ds.latitude <= -24.5)
good = ds.where(box & (ds.xco2_quality_flag == 0) & ds.xco2.notnull(), drop=True)
print(f"{n} soundings -> {int(box.sum())} in the box -> {good.sizes['sounding']} after quality control")

df = good.to_dataframe().reset_index(drop=True)                  # hand over to pandas
print(df.head(3).round(3).to_string())
print("hours covered:", df.time.dt.hour.min(), "to", df.time.dt.hour.max())
```

Output

```
without masking, fill values hide in the data: 51 | mean = -25100.4 ppm  <- nonsense
with masking, they become NaN: 51 | mean = 410.005 ppm
2000 soundings -> 433 in the box -> 287 after quality control
         xco2  xco2_quality_flag   latitude  longitude                         time
0  408.998993                0.0 -25.650999  27.773001 2023-06-01 00:09:40.177717667
1  410.455994                0.0 -24.796000  30.983000 2023-06-01 00:23:11.175946519
2  409.286987                0.0 -27.518000  28.405001 2023-06-01 00:28:17.488452498
hours covered: 0 to 23
```

The first two lines of output are the most important in the chapter. With masking off, the file's “no data” code (`-999999`) sits among the real numbers, and the mean comes out as **−25,100 ppm**. You would spot that. The dangerous case is a fill value that is *plausible*, for example 0 in a rainfall field or 255 in an 8-bit mask, which silently drags every average. Always read the variable's `_FillValue` and valid-range attributes, and let the library convert it to NaN.

> **Three silent data errors**
>
> 1. **Fill values read as data** (above). 2. **Scale and offset ignored.** Many files store integers with `scale_factor` and `add_offset`; the true value is `raw * scale + offset`. Libraries do this for you only when asked or by default; check. 3. **Quality flags ignored or misread.** A flag of 0 is “good” in some products and “bad” in others.

Time

Time is stored as numbers (seconds since 1970, days since 1900, TAI seconds) with a **units attribute** that gives the origin. Convert it to timezone-aware UTC datetimes immediately, then keep it that way. Be aware of the difference between the *observation* time, the *file* date, and the *processing* date. For orbiting instruments, also keep the **local solar time**, because local time governs illumination, boundary-layer depth and emission patterns.

Coordinates: the part everybody gets wrong once

A **coordinate reference system (CRS)** says what the numbers mean. There are two families.

- **Geographic** CRSs (such as WGS84, EPSG:4326) give positions as longitude and latitude in degrees on an ellipsoid.

- **Projected** CRSs (such as UTM, EPSG:326xx/327xx) flatten a region onto a plane and give x, y in **metres**. Distances and areas computed on the plane are accurate within the zone for which the projection was designed.

The rules that follow:

1. Store and exchange data in geographic coordinates (EPSG:4326).
2. **Do all distance, buffer, area and nearest-neighbour calculations in a metric projected CRS appropriate to your area** (the project uses UTM zone 35S, EPSG:32735).
3. Never treat a degree as a fixed length. Never average longitudes across the date line.
4. Mind the **axis order**. EPSG:4326 is officially (latitude, longitude), but most software and GeoJSON use (longitude, latitude).

```
"""s05 - Coordinates: degrees are not distances, and axis order bites."""
import warnings
import numpy as np
from pyproj import Geod, Transformer

geod = Geod(ellps="WGS84")
for lat in (0, -26, -60):
    d = geod.inv(28, lat, 29, lat)[2] / 1000
    print(f"1 degree of longitude at latitude {lat:>3}: {d:6.1f} km")
print(f"1 degree of latitude (anywhere)      : {geod.inv(28, -26, 28, -25)[2] / 1000:6.1f} km")

to_m = Transformer.from_crs("EPSG:4326", "EPSG:32735", always_xy=True)   # WGS84 -> UTM zone 35S
x1, y1 = to_m.transform(28.0, -26.0)
x2, y2 = to_m.transform(31.0, -24.5)
planar = np.hypot(x2 - x1, y2 - y1) / 1000
true = geod.inv(28.0, -26.0, 31.0, -24.5)[2] / 1000
print(f"\nKendal-ish to Lephalale-ish: UTM plane {planar:.2f} km vs geodesic {true:.2f} km "
      f"({100 * (planar / true - 1):+.3f}%)")

# the classic trap: EPSG:4326 is officially (latitude, longitude)
careful = Transformer.from_crs("EPSG:4326", "EPSG:32735")                # no always_xy
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    print("lon,lat given to an EPSG-ordered transformer ->", careful.transform(28.0, -26.0))
print("lat,lon given to the same transformer          ->",
      tuple(round(v, 1) for v in careful.transform(-26.0, 28.0)))
print("always_xy=True, lon,lat                        ->",
      tuple(round(v, 1) for v in to_m.transform(28.0, -26.0)))

# area: a 0.1 x 0.1 degree cell is not the same size everywhere
for lat in (0, -26, -60):
    a = geod.polygon_area_perimeter([28, 28.1, 28.1, 28], [lat, lat, lat - 0.1, lat - 0.1])[0]
    print(f"0.1 x 0.1 degree cell at {lat:>3}: {abs(a) / 1e6:6.1f} km2")
```

Output

```
1 degree of longitude at latitude   0:  111.3 km
1 degree of longitude at latitude -26:  100.1 km
1 degree of longitude at latitude -60:   55.8 km
1 degree of latitude (anywhere)      :  110.8 km

Kendal-ish to Lephalale-ish: UTM plane 345.05 km vs geodesic 344.88 km (+0.048%)
lon,lat given to an EPSG-ordered transformer -> (-5100101.694150827, 14600457.859334132)
lat,lon given to the same transformer          -> (600080.8, 7123933.3)
always_xy=True, lon,lat                        -> (600080.8, 7123933.3)
0.1 x 0.1 degree cell at   0:  123.1 km2
0.1 x 0.1 degree cell at -26:  110.9 km2
0.1 x 0.1 degree cell at -60:   62.1 km2
```

The output shows the facts. A degree of longitude is 111 km at the equator, 100 km at 26°S and 56 km at 60°S. A 0.1° cell is 123 km² at the equator and 62 km² at 60°S, so averaging cells as if they were equal in size, or counting pixels as if each were the same area, is wrong at high latitude. The projected plane agreed with the true geodesic distance to 0.05 % over 345 km, so within one UTM zone the plane is safe. And the axis-order trap is real: passing longitude first to a transformer that follows the official EPSG order returned absurd coordinates with no error. `always_xy=True` removes the ambiguity.

> **Key idea**
> Pick one metric CRS for computation and one geographic CRS for storage; convert at the boundary and nowhere else.

Where to get data

Portal	What it holds	Notes
NASA Earthdata (GES DISC, LP DAAC, NSIDC, PO.DAAC)	OCO-2/3, MODIS, VIIRS, GPM IMERG, SMAP and more	Free account; authorise each archive application in your profile (the project's most common failure)
Copernicus Data Space Ecosystem	Sentinel-1, -2, -3, -5P	Free account; APIs and STAC
USGS EarthExplorer / Landsat on cloud buckets	Landsat archive	Free
Google Earth Engine	Petabyte-scale catalogue with server-side processing	Free for research use; code runs on Google's servers, so it is a different workflow
Microsoft Planetary Computer and other STAC catalogues	Cloud-hosted analysis-ready data	Query by space, time and properties
Institutional and commercial (GHGSat, Planet, Carbon Mapper data releases)	Facility-scale methane, high-resolution imagery	Licences differ; check before reuse

STAC (SpatioTemporal Asset Catalog) is a standard way to describe and search satellite data. Learn it: whichever portal you meet next will probably offer it, and it lets you ask “give me all scenes over this polygon between these dates with less than 20 % cloud” in one query.

Licences and citation

Almost all public Earth-observation data are free to use, but with a citation requirement. Record, for every product you use, its **short name, version, DOI and access date**. Versions matter: the project's downloader keeps exactly one product version, because two versions of the same day mean two slightly different algorithms and double-counted observations.

Key ideas

- Arrays in NetCDF/GeoTIFF/Zarr, tables in Parquet or a database, geometry in GeoPackage or PostGIS.
- Always inspect a file first; handle fill values, scale factors and quality flags explicitly.
- Compute in a metric projected CRS, store in EPSG:4326, and guard axis order.

- Record version, DOI and access date for every product.

Exercises

1. Modify `s04` so that the quality flag is stored as 1 = good. Add an assertion that fails loudly if the fraction of good soundings is below 5 % or above 95 %. Why is such an assertion worth having?
2. Using `s05` as a model, compute the distance between Johannesburg (28.05°E, 26.2°S) and Pretoria (28.19°E, 25.75°S) in UTM and as a geodesic. What is the difference?
3. Find a STAC catalogue and write the query you would use to list Sentinel-2 scenes over your home city with under 20 % cloud in one month. (You do not have to run it; write down the fields you would filter on.)

Chapter 5. The Python toolkit and the habits that go with it

One layer at a time

Remote-sensing code in Python is built from a small set of libraries. Learn what each is *for*, and you will rarely wonder which to reach for.

Library	It is for	The moment you reach for it
`numpy`	Fast arithmetic on arrays	Any pixel-wise or sounding-wise calculation
`pandas`	Labelled tables, grouping, time series	Soundings, bins, events: anything that is rows with columns
`xarray`	Labelled N-dimensional arrays (dimensions with names)	NetCDF files, time×lat×lon cubes
`scipy`	Statistics, interpolation, spatial trees, signal processing	Distributions, KD-trees, filtering
`scikit-learn`	Clustering and machine learning	DBSCAN, random forests for land cover
`scikit-image`	Image processing	Thresholds (Otsu), morphology, filtering
`pyproj`	Coordinate transforms and geodesics	Degrees to metres and back
`shapely`, `geopandas`	Geometry objects and geometry tables	Points in polygons, buffers, spatial joins
`rasterio` (not installed in this book's environment, but standard)	Reading and writing GeoTIFFs with their georeferencing	Optical rasters
`matplotlib`, `folium`	Static figures; interactive web maps	Everything you show
`sqlalchemy`, `geoalchemy2`	Talking to PostgreSQL/PostGIS	Persisting results
`earthaccess`	NASA Earthdata search and download	Getting OCO-2, MODIS, IMERG files

Five habits that separate reliable code from fragile code

1. Vectorise, but know when a loop is fine

Replacing a Python loop with array operations is often 100× faster. Do it when the data are large. Do not do it when the loop runs over *passes* or *files*, where there are a hundred iterations and clarity matters more. The project loops over overpasses (about 137 in the synthetic set), and vectorises over the thousands of soundings within each.

2. Keep a tidy table: one row per observation

The project's central object is a table with one row per sounding and columns for time, position, value, uncertainty, quality flag and pass id. Everything later (binning, background, events) is a `groupby` on that table. A tidy table makes each step small, testable and inspectable. It also lets you add a column without rewriting code.

3. Keep everything, then flag; do not delete

When the quality filter removes 70 % of soundings, **record what was removed and why** (the project's `apply_qc` returns a report). When a bin fails a threshold, keep it with `flagged=False` rather than dropping it. The non-flagged bins are the denominator of every rate you will compute. A pipeline that deletes non-detections cannot tell you a detection *frequency*.

4. Pure functions with explicit inputs

A function that takes a table and a parameters object and returns a new table is easy to test, rerun and reason about. A function that reads module-level variables, writes files mid-way and mutates its input is not. The project's `Params` dataclass is *frozen* so a function cannot quietly change a setting; you create a modified copy instead.

```
@dataclass(frozen=True)
class Params:
    """Analysis parameters. Frozen so a run can be reproduced from its Params."""

    # --- quality control ------------------------------------------------
    keep_modes: tuple = (0, 1)        # operation_mode: 0 nadir, 1 glint (2 target, 3 transition dropped)

    # --- segmenting soundings into overpasses ---------------------------
    pass_gap_s: float = 300.0         # time gap that starts a new pass (a pass over the box lasts ~1 min)
    min_soundings: int = 150          # good soundings needed for a usable pass
    min_far_soundings: int = 60       # far-from-source soundings needed to define a background
    min_extent_km: float = 100.0      # along-track length needed (short clips cannot define a background)
```

5. Units and coordinates are part of the name

Write `dist_source_km`, `sigma_ppm`, `x_m`, `t_utc`. A column called `dist` is a bug waiting to happen. Do the same for CRS: `lon`/`lat` are degrees, `x_m`/`y_m` are projected metres.

A nearest-source search, two ways

Distance to the nearest source for every sounding is the commonest spatial query in this field. Here is the naive way and the right way, with timing, and then a point about chaining that you will need in Chapter 10.

```
"""s11 - Nearest-source search with a KD-tree, and single-linkage clustering ('chaining')."""
import time
import numpy as np
from scipy.spatial import cKDTree
from sklearn.cluster import DBSCAN

rng = np.random.default_rng(4)
sources = rng.uniform(0, 400_000, (17, 2))            # 17 sources in a 400 km square (metres)
points = rng.uniform(0, 400_000, (200_000, 2))        # 200 000 soundings

t0 = time.perf_counter()
d_brute = np.sqrt(((points[:, None, :] - sources[None, :, :]) ** 2).sum(axis=2)).min(axis=1)
t_brute = time.perf_counter() - t0
t0 = time.perf_counter()
d_tree, idx = cKDTree(sources).query(points)
t_tree = time.perf_counter() - t0
print(f"same answer: {np.allclose(d_brute, d_tree)}  | brute force {t_brute:.3f} s, KD-tree {t_tree:.3f}
s")
print(f"share of soundings within 40 km of a source: {100 * (d_tree <= 40_000).mean():.1f}%")

# Chaining: six sources 25 km apart in a line, eps = 30 km, min_samples = 1  -> ONE complex 125 km long
line = np.column_stack([np.arange(6) * 25_000.0, np.zeros(6)])
labels = DBSCAN(eps=30_000, min_samples=1).fit_predict(line)
```

```
print("\nsources 25 km apart, eps 30 km  ->", labels, "(one complex)")
labels = DBSCAN(eps=20_000, min_samples=1).fit_predict(line)
print("same sources, eps 20 km         ->", labels, "(six separate complexes)")
pts = np.vstack([line, [[150_000.0, 100_000.0]]])
print("add a lone source 100 km away   ->", DBSCAN(eps=30_000, min_samples=1).fit_predict(pts))
```

Output

```
same answer: True  | brute force 0.152 s, KD-tree 0.042 s
share of soundings within 40 km of a source: 39.2%

sources 25 km apart, eps 30 km  -> [0 0 0 0 0 0] (one complex)
same sources, eps 20 km         -> [0 1 2 3 4 5] (six separate complexes)
add a lone source 100 km away   -> [0 0 0 0 0 0 1]
```

Both methods give identical answers. The KD-tree is faster even with 17 sources, and the gap widens enormously as sources or points grow: the brute-force method needs a table of size (points × sources) in memory, which for 200,000 points and 5,000 facilities is 8 GB and will crash your machine, while the tree needs almost nothing.

Memory, size and when to scale up

A year of OCO-2 Lite files for the whole globe is hundreds of gigabytes, but the project reads them one at a time and keeps only the soundings inside the study box, a thousandth of the data. The pattern is general: **filter early, close files promptly, and never concatenate global arrays you do not need.** When data outgrow memory:

- use `xarray` with `dask` to read lazily in chunks;
- store intermediate results as Parquet, which is compressed, typed and column-oriented;
- or let a cloud service (Google Earth Engine, Planetary Computer) do the heavy computation next to the data.

Reproducible environments

Record versions. At minimum keep a `requirements.txt` with lower bounds (as the project does), and ideally a lock file. State the Python version in the README. The project was tested on Python 3.13 with both pandas 2.3 and 3.x because a pandas feature used at first (`include_groups`) existed only from 2.2, and a user on an older install would have hit a crash. If you only test on your own machine, you only know your machine works.

Key ideas

- Learn each library by its job; combine them rather than stretching one.
- One tidy table per observation, units in the names, pure functions, frozen parameters.
- Keep non-detections: they are the denominator of every rate.
- Filter early; use trees, not distance matrices.
- Test on more than one environment before saying “it works”.

Exercises

1. Rewrite the nearest-source query in `s11` using `scipy.spatial.cKDTree.query` with `k=2` to get the nearest *and* second-nearest source. For how many soundings is the second-nearest source also within 40 km?

2. Take a small table of soundings and write a pure function `add_pass_id(df, gap_s=300)` that starts a new pass at every time gap over `gap_s`. Test it on a hand-made table where you know the answer.
3. Why does the project keep unflagged bins, and what would you lose if it deleted them?

Chapter 6. Raster band maths, thresholds and honest validation

Not every remote-sensing project is about plumes. Most are about **images**: grids of pixels with several spectral bands. This chapter teaches the core image toolkit through a flood-mapping example, because floods, like fire scars and crop stress, are mapped by turning band arithmetic into a yes/no decision, and then asking how wrong that decision is.

Normalised-difference indices

The commonest idea in optical remote sensing is the **normalised difference** of two bands:

```
ND(a, b) = (a - b) / (a + b)        always between -1 and +1
```

Normalising by the sum cancels overall brightness, so shadow and sun angle matter less. Choose the two bands so that your target is high in one and low in the other.

Index	Formula	What it highlights
NDVI	(NIR − Red) / (NIR + Red)	Green vegetation: high; water: low or negative
NDWI (McFeeters)	(Green − NIR) / (Green + NIR)	Open water: positive; vegetation: negative
MNDWI (Xu)	(Green − SWIR) / (Green + SWIR)	Water, with better suppression of built-up areas
NBR	(NIR − SWIR) / (NIR + SWIR)	Burn scars: change in NBR before and after a fire shows severity

A flood-mapping experiment with a known answer

The listing builds a 240 × 240 pixel scene whose true water extent is known: a meandering river, a flooded floodplain, vegetation, bare soil, and a dark shadow patch placed on purpose. Because we know the truth, we can score each method. On a real scene you would score it against a reference you trust, such as a manual interpretation or a very high-resolution image.

```
"""s06 - Band maths for water: NDVI, NDWI, MNDWI, Otsu threshold, and honest validation."""
import numpy as np
from skimage.filters import threshold_otsu

rng = np.random.default_rng(3)
N = 240
yy, xx = np.mgrid[0:N, 0:N]
river = np.abs(yy - (120 + 30 * np.sin(xx / 28))) < 5                    # permanent water
flood = (np.abs(yy - (120 + 30 * np.sin(xx / 28))) < 24) & (yy > 100)      # flooded low ground
truth_water = river | flood
veg = (~truth_water) & (np.sin(xx / 17) * np.cos(yy / 23) > -0.2)       # vegetation, else bare soil
shadow = np.zeros((N, N), bool)
shadow[40:60, 150:190] = True                                            # a dark cloud/terrain shadow

# surface reflectance by class: green, red, nir, swir
BANDS = {"water": (0.06, 0.05, 0.03, 0.02), "veg": (0.08, 0.04, 0.45, 0.20),
         "soil": (0.15, 0.20, 0.25, 0.30), "shadow": (0.03, 0.03, 0.03, 0.03)}
cube = np.zeros((4, N, N))
for name, mask in (("soil", np.ones((N, N), bool)), ("veg", veg), ("water", truth_water), ("shadow",
shadow)):
    for b in range(4):
        cube[b][mask] = BANDS[name][b]
```

```
cube += rng.normal(0, 0.012, cube.shape)
green, red, nir, swir = np.clip(cube, 1e-3, None)

def nd(a, b):                       # normalised difference: (a - b) / (a + b), in [-1, 1]
    return (a - b) / (a + b)

ndvi, ndwi, mndwi = nd(nir, red), nd(green, nir), nd(green, swir)
print(f"NDVI  by class: water {ndvi[river].mean():+.2f}  vegetation {ndvi[veg].mean():+.2f}")
print(f"NDWI  by class: water {ndwi[river].mean():+.2f}  vegetation {ndwi[veg].mean():+.2f}")

def score(pred, truth):
    tp, fp, fn = (pred & truth).sum(), (pred & ~truth).sum(), (~pred & truth).sum()
    return tp / (tp + fp), tp / (tp + fn), tp / (tp + fp + fn)

PIXEL_KM2 = (10 * 10) / 1e6        # Sentinel-2: 10 m pixels
print(f"true flooded+river area: {truth_water.sum() * PIXEL_KM2:.2f} km2\n")
print(f"{'index':>7} {'threshold':>9} {'precision':>9} {'recall':>7} {'IoU':>6} {'area km2':>9}")
for name, idx in (("NDWI", ndwi), ("MNDWI", mndwi)):
    thr = threshold_otsu(idx)       # picks the split that best separates two modes in the histogram
    pred = idx > thr
    pr, rc, iou = score(pred, truth_water)
    print(f"{name:>7} {thr:9.2f} {pr:9.3f} {rc:7.3f} {iou:6.3f} {pred.sum() * PIXEL_KM2:9.2f}")

pred = mndwi > threshold_otsu(mndwi)
print(f"\npixels in the shadow patch called water by MNDWI: {100 * pred[shadow].mean():.0f}%")
```

Output

```
NDVI  by class: water -0.27  vegetation +0.82
NDWI  by class: water +0.35  vegetation -0.68
true flooded+river area: 0.99 km2

  index threshold precision  recall    IoU  area km2
   NDWI     -0.10     0.950   0.991  0.942      1.03
  MNDWI      0.06     0.963   0.983  0.947      1.01

pixels in the shadow patch called water by MNDWI: 46%
```



Figure 6.1. Left to right: false-colour scene, MNDWI, water mask after Otsu thresholding, and where the mask is wrong (red = false water, blue = missed water). The shadow patch (top right) is called water.

What to read from the output

- Both indices find the water almost perfectly (IoU about 0.94). That is the easy part.
- The areas are close to truth (about 1.0 km²). A small bias of a few percent is normal; **report area with an uncertainty, not as a bare number**.
- The last line is the lesson: **46 % of the shadow patch is called water**. Shadows are dark in every band, so a normalised difference of two dark numbers is near zero, and noise pushes it

across the threshold. The mask has high precision overall because the shadow is a small share of the scene, but locally it is badly wrong. A flood map without a shadow mask is wrong in the mountains, near tall buildings and under cloud shadow.

Thresholds: where should the line go?

Most indices need a threshold. Choices, from simple to robust:

1. **A fixed value** from the literature (for example NDWI > 0). Fast, but the right value shifts with sensor, atmosphere and region.
2. **Otsu's method** picks the threshold that best separates two modes in the histogram. It adapts to each scene, and works when water covers a meaningful share of pixels. If water is a tiny fraction the histogram is not bimodal and Otsu can fail badly.
3. **Local or hierarchical thresholds** applied tile by tile, then checked.
4. **A trained classifier** (random forest, U-Net) when the simple index is not enough.

Whichever you choose, **report the threshold** and test the sensitivity of your answer to it: move the threshold ±0.1 and see how much the flooded area changes.

Scoring a map: precision, recall and IoU

For a binary map compare predicted and true pixels and count:

```
TP = predicted water and truly water       FP = predicted water but dry (false alarm)
FN = missed water                          TN = correctly dry

precision = TP / (TP + FP)      of what I called water, how much was right
recall    = TP / (TP + FN)      of the true water, how much did I find
IoU       = TP / (TP + FP + FN) overlap of prediction and truth
```

Never report only overall accuracy: when water is 5 % of the scene, a map that says “dry everywhere” is 95 % accurate and useless. Report precision, recall and IoU, and look at the map of errors.

> **Where the truth comes from matters more than the formula**
>
> The numbers above are high because the truth is simulated with the same simple reflectances used to build the scene. Real validation data are noisy, may be older than the image, and may have the same blind spots as the method. A real accuracy figure needs an independent reference, a sampling design for choosing validation points, and a confidence interval. For area estimates and map accuracy, the standard reference is *Olofsson et al. (2014), “Good practices for estimating area and assessing accuracy of land change”, Remote Sensing of Environment*, which shows that counting classified pixels gives biased areas and how to correct that. Read it before publishing any area number.

Mixed pixels, clouds and shadows

A pixel is not a point; it averages everything inside it. At 10 m a pixel on a river bank is half water and half soil, and the index is somewhere in the middle. Plan for it:

- **Mixed pixels** blur the boundary. Area estimates from a threshold are biased in a direction that depends on the threshold.
- **Cloud and cloud shadow** must be masked. Satellite providers supply cloud masks; use them, and check them by eye.
- **Terrain shadow** can be predicted from a digital elevation model and the Sun's position, then masked.

- **Sun glint** on water can make water bright, not dark.

Radar for floods, in words

Optical sensors cannot see through the cloud that comes with floods. Synthetic aperture radar can. Smooth open water reflects the radar beam away from the sensor like a mirror, so it returns very little energy (dark in the image). A common workflow is: take a radar image before the flood and one during, reduce speckle noise by filtering, threshold the backscatter (or the change between the two dates) to extract water, remove areas where radar shadow or smooth surfaces such as tarmac and sand cause false water, and mask permanent water using a reference such as the JRC Global Surface Water dataset so that only the **flood extent** remains. The scoring tools above apply unchanged.

Time: composites and change

A single clear image may not exist. Two standard answers:

- **Compositing**: for each pixel take the median (or another statistic) over a time window of cloud-free observations. Medians are robust to clouds and outliers.
- **Change detection**: compute the same index on a before and an after composite and map the difference. Floods, fire and clearing all show up as a change.

Key ideas

- Normalised differences cancel brightness; choose bands where the target differs.
- Thresholds are choices: report them and test sensitivity.
- Score with precision, recall and IoU against an independent reference; look at the error map.
- Shadows and mixed pixels are the standard failure modes; mask them.
- Radar sees through cloud; optical shows spectral detail. Floods often need both.

Exercises

1. Run `s06` with the shadow patch removed. What do precision and recall become? Then move the Otsu threshold by ±0.1 and record the area change.
2. Add a shadow mask by also requiring NIR reflectance above 0.05. What does it do to the shadow patch, and to the real water pixels (water is also dark in NIR)? What does this tell you about masks?
3. Compute NDVI for the scene. Where is it highest? What would a before/after NDVI difference show after a crop fire?

Chapter 7. Time series, climatology and anomalies

Drought, rainfall extremes, vegetation stress and unusual heat are all defined the same way: **how different is this from what is normal for this place and time of year?** That sentence has three parts, and each hides a decision.

The recipe

1. **Choose a baseline period** that defines “normal” (commonly 30 years, or the whole record if it is short).
2. **Compute a climatology**: the typical value for each calendar month or day of year.
3. **Subtract** (or standardise) to get an anomaly.
4. **Decide what counts as unusual** and say so in terms others can check.

The same four steps appear in the CO_2 project, with “this overpass” standing in for “this month” and “the far field of the same pass” standing in for the climatology. Anomaly thinking transfers directly between a rainfall record and a plume.

Seasonality first

Subtracting a single annual mean from monthly data leaves the seasonal cycle in the residual, and the “anomaly” is mostly the season. Always compute the climatology **per calendar month**. A related trap is **trend**: if there is a trend, the baseline period matters, and anomalies relative to an old baseline will be biased. Decide explicitly whether you want anomalies relative to a fixed baseline (good for climate monitoring) or relative to a recent running climatology (good for detecting short events).

A z-score is not enough when data are skewed

The z-score, (value − mean) / standard deviation, assumes a roughly symmetric distribution. Rainfall is not symmetric: it cannot go below zero, it is skewed and, in dry months, full of zeros. For such data a z-score gives misleading probabilities. The standard fix is the **Standardised Precipitation Index (SPI)**: fit a gamma distribution to the record for each calendar month, compute the percentile of the observed value, then convert that percentile to the equivalent normal score. The result is a number you can read like a z-score (−1.5 is a roughly 1-in-15 dry month) even though rainfall is not normal.

```
"""s07 - Time-series anomalies: climatology, z-score, and the standardised precipitation index (SPI)."""
import numpy as np
import pandas as pd
from scipy import stats

rng = np.random.default_rng(5)
years = np.arange(1985, 2025)
months = np.arange(1, 13)
# invented monthly rainfall (mm): strong summer season, gamma-distributed so it is skewed and >= 0
mean_mm = 15 + 95 * np.exp(-0.5 * ((months - 1.5 + 6) % 12 - 6) ** 2 / 2.2 ** 2)
rows = []
for y in years:
    drought = 0.45 if y in (2015, 2016) else 1.0                  # two bad years
    for m in months:
        shape = 2.2
        rows.append((y, m, rng.gamma(shape, drought * mean_mm[m - 1] / shape)))
df = pd.DataFrame(rows, columns=["year", "month", "mm"])

# 1) simple z-score against the calendar-month climatology
clim = df.groupby("month")["mm"].agg(["mean", "std"])
df = df.join(clim, on="month")
df["z"] = (df["mm"] - df["mean"]) / df["std"]
```

```
# 2) SPI: fit a gamma distribution per calendar month, then map the percentile onto a normal curve
def spi_for_month(g):
    a, loc, scale = stats.gamma.fit(g["mm"], floc=0)
    return pd.Series(stats.norm.ppf(stats.gamma.cdf(g["mm"], a, loc, scale)).clip(-3.5, 3.5),
index=g.index)
df["spi"] = df.groupby("month", group_keys=False).apply(spi_for_month)

dry = df[df.month.isin([6, 7, 8])]                                  # the dry season
print("Skewness of January rainfall (0 = symmetric):", round(stats.skew(df[df.month == 1].mm), 2))
print("share of months with z < -1.5   :", f"{100 * (df.z < -1.5).mean():.1f}%",
      "  with SPI < -1.5:", f"{100 * (df.spi < -1.5).mean():.1f}%",
      "  (a normal variable gives 6.7%)")
print("dry-season share, z < -1.5       :", f"{100 * (dry.z < -1.5).mean():.1f}%",
      "  SPI < -1.5:", f"{100 * (dry.spi < -1.5).mean():.1f}%")
yr = df.groupby("year")[["z", "spi"]].mean()
print("\nwettest and driest years by mean SPI:")
print(yr.sort_values("spi").iloc[[0, 1, -1]].round(2).to_string())
```

Output

```
Skewness of January rainfall (0 = symmetric): 0.87
share of months with z < -1.5   : 0.2%   with SPI < -1.5: 6.5%   (a normal variable gives 6.7%)
dry-season share, z < -1.5       : 0.8%   SPI < -1.5: 5.8%

wettest and driest years by mean SPI:
         z   spi
year
2016 -0.79 -0.94
2015 -0.76 -0.85
1987  0.63  0.62
```

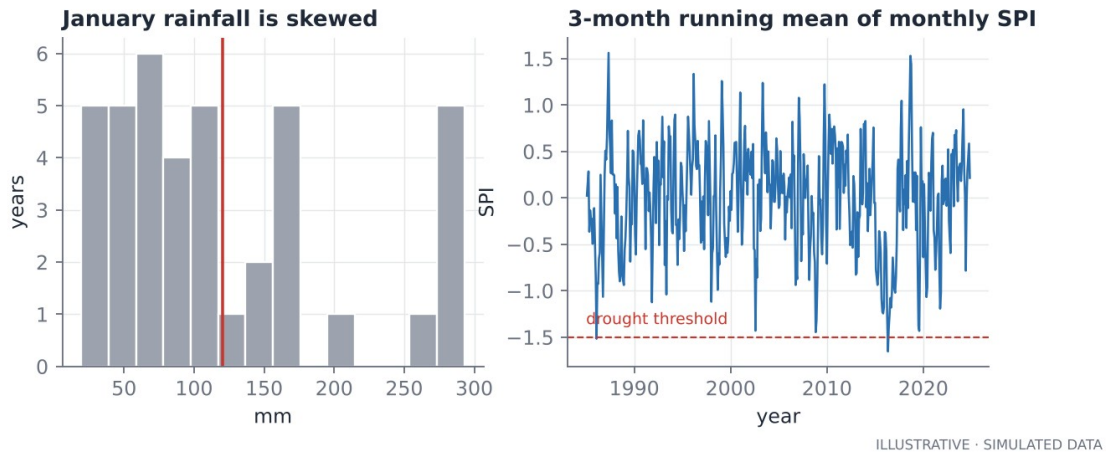


Figure 7.1. Left: January rainfall is skewed, not bell-shaped. Right: running mean of the monthly SPI with the two planted drought years standing out (2015–2016).

Reading the output

The plain z-score flags **0.2 %** of months below −1.5, while the SPI flags 6.5 %, close to the 6.7 % a normal variable would give. Gamma-distributed rainfall has a short lower tail, so its standard-deviation-based z-score rarely reaches −1.5 even in a genuinely bad month. If you used the z-score you would almost never report a drought, and the planted drought years would look milder than they were (mean z of about −0.8 against mean SPI of about −0.9). The SPI treats dry and wet extremes on the same probability scale.

> **About “SPI-3”**
> The listing uses a monthly SPI and the figure smooths it. The operational SPI-3 (or SPI-6, SPI-12)

first **accumulates rainfall over 3 (or 6, 12) months** and then fits the distribution, which captures the lagged effect of rainfall on soil moisture and reservoirs. For real drought monitoring use the accumulated version.

Choosing the quantity to monitor

Hazard	Common variable	Notes
Meteorological drought	SPI from rainfall (IMERG, CHIRPS, gauges)	Rainfall only; ignores evaporation
Agricultural drought	Vegetation anomalies (NDVI, VCI), soil moisture	Vegetation responds with a lag; crop calendar matters
Heat extremes	Land surface temperature anomaly	Clear-sky bias in thermal data
Fire risk	Fuel dryness, vegetation indices	Needs local calibration

The **Vegetation Condition Index** rescales each pixel's NDVI between its own historical minimum and maximum for that calendar month, so a value near 0 means “as poor as it has ever been here”. It is a cheap, effective drought indicator and a good first project for a portfolio.

Rainfall products and what to distrust

Gridded rainfall from satellite (GPM IMERG, CHIRPS and others) merges passive microwave, infrared cloud-top temperature and, in some products, gauge data. Remember:

- Satellite rainfall is **an estimate**. It is typically good for accumulations over days to months and large areas, and weaker for short, intense, local storms and in mountains.
- Products differ. Compare at least two before drawing conclusions.
- Where rain gauges exist, **validate against them** with bias, correlation and root-mean-square error, per season. A single “correlation 0.8” hides large seasonal biases.
- Gauges themselves are point measurements. A 10 km pixel and a gauge are different quantities.

Gaps, missing data and compositing

Real time series have holes. Do not interpolate silently. Options: leave the gaps and compute statistics that tolerate them; fill short gaps with a stated method and flag the filled values; or aggregate to a coarser time step (weekly, monthly) with a minimum-observations rule. Always say how many observations contributed to each aggregated value.

Significance and trends

If someone asks “is the trend real?” they are asking a statistics question that Chapter 8 answers: you need a model of the noise, and the noise here is autocorrelated (this month resembles last month), so naive significance tests are over-optimistic. A robust trend estimator (Theil–Sen slope) with a test that accounts for autocorrelation (modified Mann–Kendall) is the standard remote-sensing answer.

Key ideas

- Anomaly = value relative to a baseline for the same place and season. State the baseline.
- Compute climatologies per calendar month, not one mean.
- For skewed data (rainfall) use percentile-based scores such as the SPI, not a plain z-score.
- Validate satellite products against independent ground data and say how.

- Autocorrelation makes naive trend tests over-confident.

Exercises

1. Change the planted drought in `s07` from 45 % to 70 % of normal rainfall. At what drought severity do the z-score and SPI start to disagree about whether it was a drought?
2. Implement SPI on 3-month accumulated rainfall (use `pandas.rolling(3).sum()` before fitting). How do the 2015–2016 values change?
3. Sketch how you would compute the Vegetation Condition Index for 20 years of monthly NDVI. What would you do about months with missing data?

Chapter 8. Signal, noise and uncertainty

This is the most important chapter in the book. Every detection, every trend, every emission estimate is a claim that a signal stands above noise, and the entire craft is in knowing how big the noise really is. Most errors in published-looking remote-sensing work are not coding errors. They are an underestimated noise level.

Two kinds of error

- **Random error** differs from one observation to the next and averages down: collect more data, and the mean becomes steadier. Detector noise is the standard example.
- **Systematic error** (bias) is the same, or slowly varying, across many observations and does **not** average down. A calibration offset, an aerosol effect that changes smoothly over a region, a surface-brightness artefact: more soundings give you a more precise wrong answer.

Precision is how tightly repeated measurements cluster. Accuracy is how close they are, on average, to the truth. You can have high precision and poor accuracy. Remote sensing is a constant fight with the second.

Averaging and the square-root law

If errors are independent, with standard deviation σ each, the mean of n observations has standard deviation

```
SE = sigma / sqrt(n)
```

This is the **standard error**. Four times as many soundings halve it. It is why the project averages soundings into 5 km bins before looking for plumes: 20 soundings per bin shrink the noise by a factor of about 4.5.

A **z-score** divides the observed mean by its standard error: z = mean / SE. Under pure noise z is roughly standard normal, so z ≥ 3 should happen about 0.13 % of the time. That promise is the entire basis for a rule like “flag a bin if z is at least 3”.

Why the promise fails: correlated errors

The promise holds only if the errors are **independent**. Satellite errors are not. Neighbouring soundings share the same aerosol layer, the same surface type, the same slowly varying calibration and retrieval error. When errors are positively correlated, averaging helps much less than the square-root law claims.

For a first-order autoregressive error with correlation ρ between neighbours, the number of effectively independent observations is approximately

```
n_eff = n * (1 - rho) / (1 + rho)
```

and the true spread of the mean is larger than the naive one by sqrt((1 + ρ) / (1 − ρ)). The next listing shows it with pure-noise simulations: no plume exists, so every flag is a false alarm.

```
"""s08 - Why the textbook standard error lies when errors are correlated."""
import numpy as np

rng = np.random.default_rng(11)

def false_alarm_rate(rho, n=20, trials=40000, z_cut=3.0):
    """Pure noise, no plume. Soundings along a track have correlated errors (AR(1))."""
    x = np.empty((trials, n))
    x[:, 0] = rng.normal(size=trials)
    for t in range(1, n):
```

```
        x[:, t] = rho * x[:, t - 1] + np.sqrt(1 - rho ** 2) * rng.normal(size=trials)
    z = x.mean(axis=1) / (1.0 / np.sqrt(n))        # the naive z-score: mean / (sigma / sqrt(n))
    return (z >= z_cut).mean(), z.std()

print(f"{'rho':>5} {'z spread':>9} {'false alarms (z>=3)':>20}   naive promise: 0.13%")
for rho in (0.0, 0.3, 0.6, 0.9):
    fa, zs = false_alarm_rate(rho)
    print(f"{rho:5.1f} {zs:9.2f} {100 * fa:19.2f}%")

# Effective sample size: n * (1 - rho) / (1 + rho)
for rho in (0.3, 0.6, 0.9):
    print(f"rho={rho}: 20 soundings are worth {20 * (1 - rho) / (1 + rho):4.1f} independent ones")

# Second trap: bins inside one pass share an offset, so the PASS is the unit of replication.
n_pass, n_bin = 40, 30
offset = rng.normal(0, 0.5, n_pass)                           # shared by every bin of a pass
vals = offset[:, None] + rng.normal(0, 0.2, (n_pass, n_bin))  # bin means
flat = vals.ravel()
naive_se = flat.std(ddof=1) / np.sqrt(flat.size)
boot_pass = [vals[rng.integers(0, n_pass, n_pass)].mean() for _ in range(3000)]
boot_bins = [rng.choice(flat, flat.size).mean() for _ in range(3000)]
print(f"\nSE of the grand mean: naive {naive_se:.3f} | bootstrap over bins {np.std(boot_bins):.3f} "
      f"| bootstrap over passes {np.std(boot_pass):.3f}")
print(f"treating bins as independent makes the error bar {np.std(boot_pass) / np.std(boot_bins):.1f}x too
narrow")
```

Output

```
  rho  z spread  false alarms (z>=3)   naive promise: 0.13%
  0.0      1.00                0.14%
  0.3      1.35                1.29%
  0.6      1.92                6.11%
  0.9      3.32               18.48%
rho=0.3: 20 soundings are worth 10.8 independent ones
rho=0.6: 20 soundings are worth  5.0 independent ones
rho=0.9: 20 soundings are worth  1.1 independent ones

SE of the grand mean: naive 0.014 | bootstrap over bins 0.015 | bootstrap over passes 0.072
treating bins as independent makes the error bar 4.9x too narrow
```

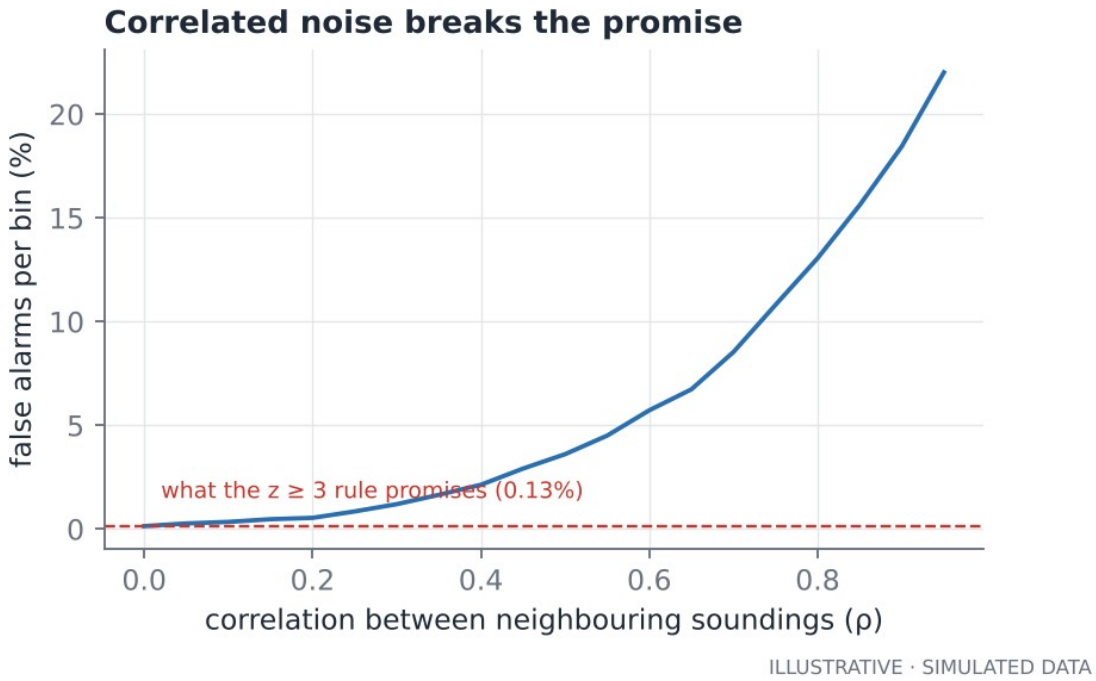


Figure 8.1. Fraction of pure-noise bins that pass the naive z ≥ 3 test, as the correlation between neighbouring soundings grows. The promised 0.13 % is the dashed line.

What the numbers say

- With independent errors (ρ = 0) the rule delivers its promise: 0.14 % false alarms.
- With a modest ρ = 0.3 it is already ten times worse (1.3 %).
- At ρ = 0.6 it is 6 %, and at ρ = 0.9 nearly one bin in five is a false alarm. Twenty soundings are then worth about one independent observation.

The second half of the listing shows a different version of the same disease. Bins within one overpass share an offset (the pass's own background error). If you treat all 1,200 bins as independent, the grand mean looks known to ±0.014 ppm, but the honest answer, found by resampling whole passes, is ±0.072: the error bar was **five times too narrow**. The lesson is general: **find the real unit of replication.** Here it is the pass, not the bin or the sounding.

> **Key idea**
> The unit you can resample independently is the unit that carries the uncertainty. Soundings in a bin, and bins in a pass, are not independent. Passes, on different days with different atmospheres, are.

Robust statistics: when data are not well behaved

Real data have outliers: a cloud edge, a bad retrieval, a spike. The mean and standard deviation are fragile; one wild value moves them. Robust alternatives:

- the **median** instead of the mean;
- the **median absolute deviation**, scaled: robust σ = 1.4826 × median(|x − median(x)|), which estimates the standard deviation of a normal distribution while ignoring outliers.

The project uses both for the background and the noise estimate, and its tests include a case with 2 % wild outliers that would wreck a mean-based method. In clean data (as in the simulations here) robust and ordinary estimates agree, which is what you want: the robust method costs almost nothing when data are good and saves you when they are not.

The bootstrap: uncertainty without formulas

When no clean formula exists for the uncertainty of a statistic (a ratio of rates, a median, a profile), **resample and recompute**. Draw, with replacement, a new dataset of the same size from your data; compute the statistic; repeat a few thousand times; the spread of the results is the uncertainty. Percentiles of that spread give a confidence interval.

Two rules make the bootstrap honest. Resample the **independent units** (passes, not bins), and make the statistic computation exactly the one you report. The project's control test (Chapter 10) uses this to give a confidence interval on a ratio of flag rates.

Intervals for proportions: Wilson

A detection frequency is a proportion: events observed out of passes that looked. The textbook normal-approximation interval fails at small counts and is useless at zero. The **Wilson score interval** behaves well for small samples and for 0 or n successes. Zero events in ten passes does *not* mean the true frequency is zero; it means it could plausibly be as high as 28 %, a fact Chapter 10 puts to use.

Hypothesis tests and controls

A hypothesis test asks: if there were no effect, how surprising is what I saw? The p-value is that surprise. Three cautions:

1. A tiny p-value says the effect is probably not zero. It does **not** say the effect is large, important or caused by what you think.
2. If you test many things, some pass by chance. Decide your tests *before* looking.
3. A test is only as good as its assumptions. The Fisher exact test used for the control comparison treats bins as independent, which they are not, so the p-value is **optimistic**; that is why the project also gives a bootstrap interval over passes, and why tests clamp the p-value at 10^{-300} rather than printing an impossible zero.

The most powerful tool is not a test but a **control**: apply your method to places where the answer is known to be “nothing”, and count the false alarms. If the control region flags at nearly the same rate as the target region, your detections mean nothing, whatever the p-values say. A control converts an argument into a measurement.

The error budget habit

For any quantity you report, list its error sources, give each a size, and combine them. For a flux estimate (Chapter 11) the budget has at least: instrument noise in the enhancement, background estimation error, wind error, and plume-geometry error. Seeing that wind error is 30 % while instrument noise is 10 % tells you where to spend effort. An estimate without a budget is a guess with decimals.

Key ideas

- Random error averages down; systematic error does not.
- The square-root law needs independent errors. Correlation shrinks the effective sample size.
- Find the real unit of replication and resample it.
- Use robust statistics and bootstraps when data are messy or formulas do not exist.
- Controls beat p-values: measure the false-alarm rate directly.
- Write an error budget for every reported number.

Exercises

1. In `s08`, set n = 50 per bin instead of 20 and rerun. Does the false-alarm rate at ρ = 0.6 improve? Why or why not? (Hint: n_eff.)

2. Replace the bin-level offset in the pass experiment with an offset of 0.1 instead of 0.5. How does the ratio of honest to naive error bars change? What does that say about when the problem matters?
3. Construct a small dataset of 30 numbers with one wild outlier. Compare the mean and standard deviation to the median and 1.4826·MAD, with and without the outlier.

Chapter 9. Background, anomalies and detection

The enhancement idea

A point source adds a **small, local enhancement** on top of a **large, smooth background**. You cannot see it in the raw values, where the background (about 415 ppm of CO_2, say) dominates and varies from day to day and place to place. You *can* see it if you estimate the background and subtract it. This idea, in one form or another, underlies almost every gas-plume detection from space.

```
anomaly = measured value - estimated background
```

Everything depends on the quality of that background estimate, so it deserves its own chapter.

Choosing a background

The background must represent **what the measurements would have been without the source**, at the same time and place. Candidates, from crude to careful:

1. **One global mean.** Fails whenever the background has a gradient or an offset that differs per pass.
2. **A per-pass median of far-field soundings.** Removes the pass offset; blind to gradients within the pass.
3. **A running median along the track** using only far-field soundings. Follows slow trends, and needs enough far-field soundings on both sides.
4. **A model-based background** (an atmospheric transport model plus prior fluxes). Powerful, but the model's own errors enter.
5. **A regression background** using covariates (for example surface elevation or aerosol) to explain variability that is not the source.

The *far field* is the set of soundings you trust to be free of the source: farther than some **exclusion radius** from any known source. Exclusion radius is a physical choice, not a tuning knob: it should be larger than the distance over which a plume is expected to extend, and it should be tied to the radius you later use to attribute events (the project once used 30 km for attribution and 40 km for exclusion, which orphaned 29 % of its events; see Chapter 15).

Three backgrounds on one simulated pass

The listing creates one 300 km overpass with a slow background gradient, noise of 0.8 ppm per sounding and one planted plume of 2 ppm, and tries three backgrounds. Because we planted the truth, we can measure each method's error.

```
"""s09 - Three ways to estimate the background along one overpass, and how much each one costs you."""
import numpy as np

rng = np.random.default_rng(2)
x = np.sort(rng.uniform(0, 300, 1200))                      # along-track position, km (8 footprints x
rows)
truth_bg = 410.0 + 0.004 * x + 0.8 * np.sin(x / 70)         # a slow gradient the satellite does not know
about
plume_x, plume_amp = 150.0, 2.0
plume = plume_amp * np.exp(-0.5 * ((x - plume_x) / 6) ** 2)
noise = rng.normal(0, 0.8, x.size)
y = truth_bg + plume + noise
far = np.abs(x - plume_x) > 40                              # 'far field': away from the known source

def running_median(x, y, mask, half_window=50.0):
    out = np.empty_like(x)
```

```
    for i, xi in enumerate(x):
        sel = mask & (np.abs(x - xi) <= half_window)
        out[i] = np.median(y[sel])
    return out

bg_mean = np.full_like(x, y.mean())                          # (a) one mean of everything
bg_median = np.full_like(x, np.median(y[far]))               # (b) one median of the far field
bg_run = running_median(x, y, far)                           # (c) running median of the far field

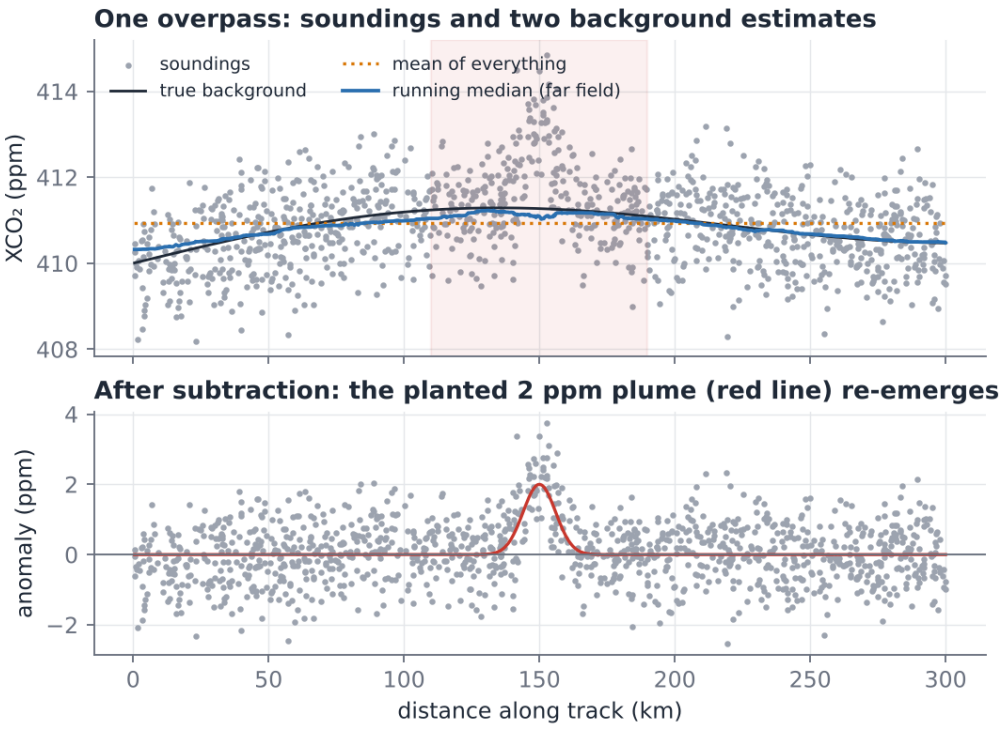
print(f"{'method':<22} {'bg RMSE (ppm)':>14} {'bias at plume (ppm)':>20}")
for name, bg in (("mean of everything", bg_mean), ("median, far field", bg_median),
                 ("running median", bg_run)):
    rmse = np.sqrt(np.mean((bg - truth_bg) ** 2))
    near = np.abs(x - plume_x) < 6                    # bias = recovered plume minus true plume, same
window
    bias = np.mean((y - bg)[near] - plume[near])
    print(f"{name:<22} {rmse:14.2f} {bias:+20.2f}")

resid = (y - bg_run)[far]
sigma = 1.4826 * np.median(np.abs(resid - np.median(resid)))
print(f"\nrobust sigma (1.4826 x MAD): {sigma:.2f} ppm   plain std: {resid.std():.2f} ppm   true noise:
0.80")
```

Output

```
method                 bg RMSE (ppm)  bias at plume (ppm)
mean of everything              0.34               +0.26
median, far field               0.36               +0.46
running median                  0.10               +0.09

robust sigma (1.4826 x MAD): 0.86 ppm   plain std: 0.82 ppm   true noise: 0.80
```



ILLUSTRATIVE · SIMULATED DATA

Figure 9.1. Top: soundings, the true background and two estimates. Bottom: after subtracting the running-median background, the planted plume (red) is recovered.

The running median has a background error of 0.10 ppm, three to four times smaller than a single mean or median (0.34 and 0.36), and its bias at the plume is +0.09 ppm against +0.26 and +0.46. That seems small, but the plume is 2 ppm and the detection floor is 1 ppm: a 0.4 ppm bias changes a 1.6 ppm plume into a 2.0 ppm one and shifts detections in and out. Remember that **in the real world there is no truth column to check against**, which is why the simulation exists.

> **The far field is not automatically clean**
>
> If there are unknown sources inside your “far field” (an unlisted facility, a city), they leak into the background and shrink or remove the apparent plume of known sources. Check by plotting the far-field anomalies for structure, and keep an *unattributed events* table so unexplained hotspots become findings, not nuisances.

Estimating the noise

Detection compares an anomaly with the scatter of the background. The project estimates that scatter from the far-field residuals using the robust σ (1.4826 × MAD) and floors it at 0.3 ppm so a quiet pass never gets an implausibly small noise. In the listing the robust estimate (0.86 ppm) is slightly above the true noise (0.80) because it also contains the background estimation error, which is what you want: it includes everything that adds scatter to the anomaly. The **reported uncertainty** in the data file is another, independent estimate; compare the two. If measured scatter is much larger than reported uncertainty, correlated errors are dominating and thresholds need to rise.

From anomalies to detections: bins and two conditions

Single soundings are noisy. The project averages anomalies into **5 km along-track bins** and flags a bin only if **both** conditions hold:

- the bin mean is at least **1 ppm**: it matters physically;
- the bin mean is at least **3 standard errors**: it is not just noise.

Adjacent flagged bins merge into one **event**. Why two conditions? The z-score alone allows a very small mean (0.5 ppm) when the noise is small, which is statistically unlikely but physically uninteresting and, as Chapter 8 showed, over-trusting. The absolute floor guards against that and against underestimated noise. The z-condition guards against flagging a 1.2 ppm mean when the noise is 1 ppm per bin.

Detection limits: how big must a plume be?

How often is a plume of a given size flagged, and how often is noise flagged? Both answers are needed. A plume detection limit is not a single number; it is a **probability curve**.

```
"""s10 - Detection limits: how big must a plume be before the two-condition rule finds it?"""
import numpy as np

rng = np.random.default_rng(21)
SIGMA, N_PER_BIN, MIN_ENH, Z_CUT = 0.8, 20, 1.0, 3.0

def simulate(amplitude, rho, trials=20000, width_bins=3):
    """A plume covers `width_bins` bins; return how often at least one bin is flagged."""
    def bin_means(extra):
        x = np.empty((trials, N_PER_BIN)); x[:, 0] = rng.normal(size=trials)
        for t in range(1, N_PER_BIN):
            x[:, t] = rho * x[:, t - 1] + np.sqrt(1 - rho ** 2) * rng.normal(size=trials)
        return SIGMA * x.mean(axis=1) + extra
    hits = np.zeros(trials, bool)
    for k in range(width_bins):
        m = bin_means(amplitude * np.exp(-0.5 * ((k - 1) / 1.0) ** 2))   # peak in the middle bin
```

```
        z = m / (SIGMA / np.sqrt(N_PER_BIN))                              # the naive z-score
        hits |= (m >= MIN_ENH) & (z >= Z_CUT)
    return hits.mean()

amps = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0]
print(f"{'plume ppm':>9} {'P(detect) rho=0':>16} {'P(detect) rho=0.5':>18}")
res = {0.0: [], 0.5: []}
for a in amps:
    for rho in (0.0, 0.5):
        res[rho].append(simulate(a, rho))
    print(f"{a:9.1f} {100 * res[0.0][-1]:15.1f}% {100 * res[0.5][-1]:17.1f}%")

print("\nrow 0.0 is the false-alarm rate per 3-bin window when BOTH conditions must hold.")
print(f"minimum mean for a flag is {MIN_ENH} ppm regardless of noise; "
      f"the z rule alone would allow {Z_CUT * SIGMA / np.sqrt(N_PER_BIN):.2f} ppm")
```

Output

```
plume ppm  P(detect) rho=0  P(detect) rho=0.5
      0.0             0.0%               0.1%
      0.5             0.2%               6.5%
      1.0            51.3%              58.9%
      1.5            99.9%              98.3%
      2.0           100.0%             100.0%
      2.5           100.0%             100.0%
      3.0           100.0%             100.0%
      4.0           100.0%             100.0%

row 0.0 is the false-alarm rate per 3-bin window when BOTH conditions must hold.
minimum mean for a flag is 1.0 ppm regardless of noise; the z rule alone would allow 0.54 ppm
```

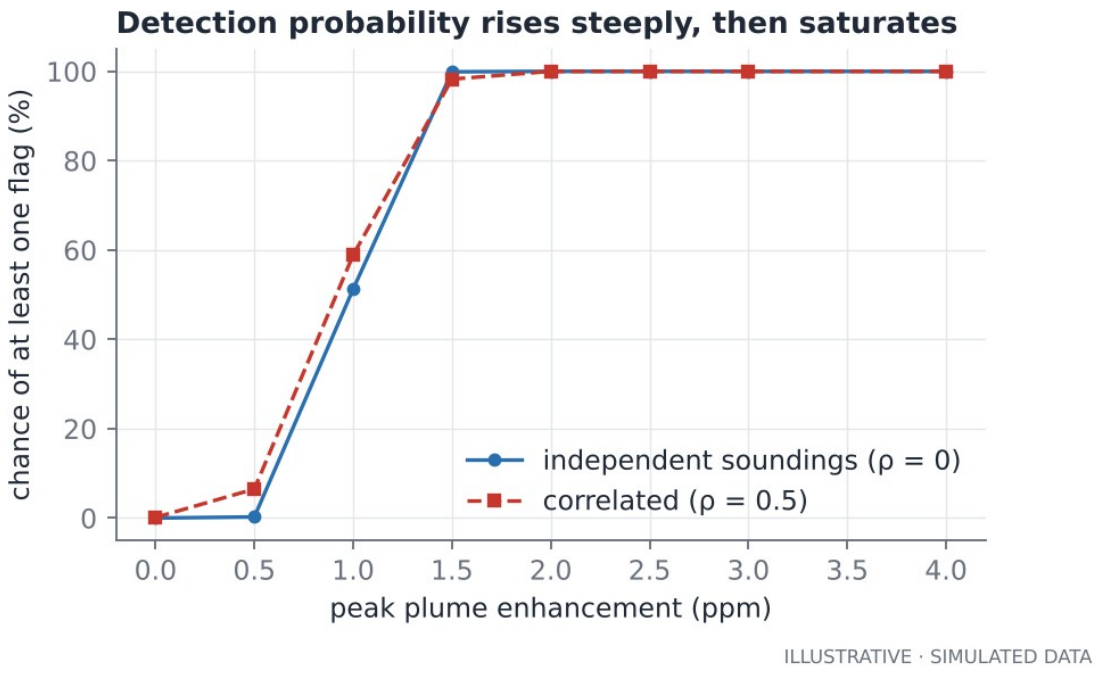


Figure 9.2. Chance that a simulated plume of given peak enhancement produces at least one flagged bin, for independent and correlated errors.

Read the table as a design review of the detection rule.

- **False alarms are essentially zero** under both noise models, because the 1 ppm floor is five or more bin-standard-errors above zero. The floor, not the z-score, is doing the protecting.

- **The 50 % detection point is at about 1 ppm**, exactly the floor. Below 0.5 ppm a plume is almost never seen; above 1.5 ppm almost always. The rule has a sharp edge, set by a single parameter.
- The cost of the floor is **sensitivity**: any source producing less than about 1 ppm at the satellite is invisible to this method, however many times it is observed. Chapter 11 turns that into an emission rate.
- Correlation barely changes the detection curve here (the floor dominates) but would change the false-alarm rate a lot if the floor were lowered. If you ever lower the threshold to find more plumes, re-measure the false-alarm rate with the control test first.

What else looks like a plume

- **Surface artefacts.** A change in surface albedo, such as a dark dam, burn scar, or bright pan, can bias the retrieval over tens of kilometres.
- **Topography.** Ridges and escarpments change surface pressure and path length.
- **Aerosol plumes** (smoke, dust) correlate with real combustion sources and with artefacts.
- **Cities.** Urban domes produce genuine, broad CO_2 enhancements that are not the nearby power plants.
- **Seasonal biosphere.** Active vegetation draws CO_2 down in the growing season, shifting backgrounds and creating gradients.

Beyond a threshold

Thresholds are the right first method because they are transparent and fast to verify. Later you can add:

- a **matched filter** that scores how well the observed spectrum matches the gas signature (Chapter 11);
- a **model-based background** from a transport model;
- **machine-learning detectors** trained on labelled plumes (promising for images, but need labelled data and an honest false-alarm measurement; Chapter 15);
- **multi-day persistence** (does the same place show up repeatedly?), which cuts false alarms because artefacts rarely repeat at the same place on different days with different geometry.

Key ideas

- Estimate the background from a far field, using a running median to follow gradients.
- Tie the exclusion radius to the attribution radius, and write down why.
- Compare measured scatter with reported uncertainty.
- Require both a physical size and a statistical significance for a flag.
- Describe detection limits as a probability curve, and measure the false-alarm rate with simulations and controls.
- Know what mimics a plume.

Exercises

1. In `s09`, add a localised retrieval artefact of +1 ppm over 20 km at x = 60. How does the detection of the real plume at 150 km change? How would you spot the artefact?
2. In `s10`, lower the floor to 0.5 ppm. What happens to the false-alarm rate at $\rho = 0$ and at $\rho = 0.5$? What does that say about relying on the z-score?
3. Write a rule that requires an event to be seen on at least two different days before it is reported. What would it do to detection probability for a source that is observed 25 times a year?

Chapter 10. Space, attribution and sampling

Questions of “where” and “which”

Having found an enhancement, you ask: *where is it, and which source does it belong to?* And, because the satellite only looks occasionally, *how often would we see it if it were there?* This chapter covers the spatial machinery and the sampling statistics that make the answer honest.

Attribution by proximity, and its limits

The simplest attribution is **nearest source within a radius**: an event within 40 km of a known facility is assigned to the nearest one. It is fast, transparent and needs no wind data. It is also easily wrong:

- the plume may be blown tens of kilometres away from its source;
- several sources may lie within the radius;
- an unlisted source may be the real cause;
- a city may sit among small plants and masquerade as them.

So treat proximity attribution as **association, not causation**. Write “a detection associated with the Kendal complex”, not “Kendal emitted X”. To go further you need wind and transport (Chapter 11).

Complexes: when sources are too close to separate

A 10 km swath and a few-kilometre plume cannot tell apart plants that are a few kilometres from each other. The project groups sources into **complexes** by single-linkage clustering (DBSCAN with `min_samples=1`): if A is within the link distance of B and B of C, all three are one complex, even if A and C are far apart. That *chaining* is a feature when you want connected groups and a hazard when a chain grows longer than you expected.

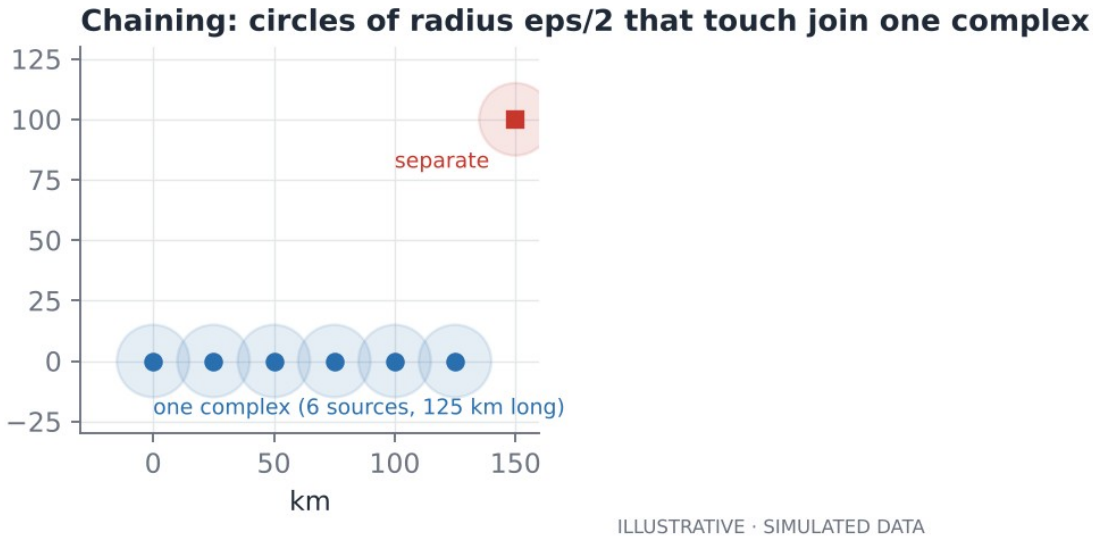


Figure 10.1. Six sources 25 km apart chain into one complex 125 km long when the link distance is 30 km; a source 100 km away stays separate.

The spatial listing from Chapter 5 shows this: with link distance 30 km, six sources 25 km apart become one complex; with 20 km they are six. Parameters that look innocent control the unit of analysis, so record them in the output.

Counting observations: the denominator

A count of events is meaningless without a count of opportunities. The project keeps **every** bin, not only flagged ones, so for each complex it can compute two numbers:

- **observed passes**: passes with at least 2 valid bins within the coverage radius of any source in the complex;
- **event passes**: those observed passes with at least one flagged bin.

The **detection frequency** is event passes divided by observed passes, and it is reported with a Wilson interval (Chapter 8). A complex with 3 event passes in 40 observed passes and one with 3 in 4 are completely different findings, although both have “3 detections”.

How many looks do you really get?

```
"""s14 - How many looks do you get? Sampling arithmetic and what it does to a detection frequency."""
import numpy as np
import sys
sys.path.insert(0, "/home/claude/highveld-co2")
from highveld_co2.attribute import wilson_ci            # the project's own function, reused

passes_per_cycle, cloud_free_share, cycles_per_year = 2.6, 0.45, 365.25 / 16
usable = passes_per_cycle * cycles_per_year * cloud_free_share
print(f"usable passes per year over a 4-degree box: ~{usable:.0f}")

rng = np.random.default_rng(1)
n_years = 10000
n_usable = rng.poisson(usable, n_years)
print(f"in 1 year: P(fewer than 10 usable passes) = {(n_usable < 10).mean():.2f}; "
      f"median {np.median(n_usable):.0f}, 5th-95th percentile {np.percentile(n_usable, 5):.0f}-
{np.percentile(n_usable, 95):.0f}")

print(f"\n{'passes n':>9} {'true freq 0.2: 95% CI':>24} {'width':>7}")
for n in (5, 10, 20, 50, 100, 400):
    k = int(round(0.2 * n)); lo, hi = wilson_ci(k, n)
    print(f"{n:9d} {lo:11.2f} - {hi:5.2f} {hi - lo:12.2f}")
lo, hi = wilson_ci(0, 10)
print(f"\n0 events in 10 passes: the frequency could still be as high as {hi:.0%}")
```

Output

```
usable passes per year over a 4-degree box: ~27
in 1 year: P(fewer than 10 usable passes) = 0.00; median 27, 5th-95th percentile 19-36

 passes n    true freq 0.2: 95% CI   width
        5        0.04 -  0.62         0.59
       10        0.06 -  0.51         0.45
       20        0.08 -  0.42         0.34
       50        0.11 -  0.33         0.22
      100        0.13 -  0.29         0.16
      400        0.16 -  0.24         0.08

0 events in 10 passes: the frequency could still be as high as 28%
```

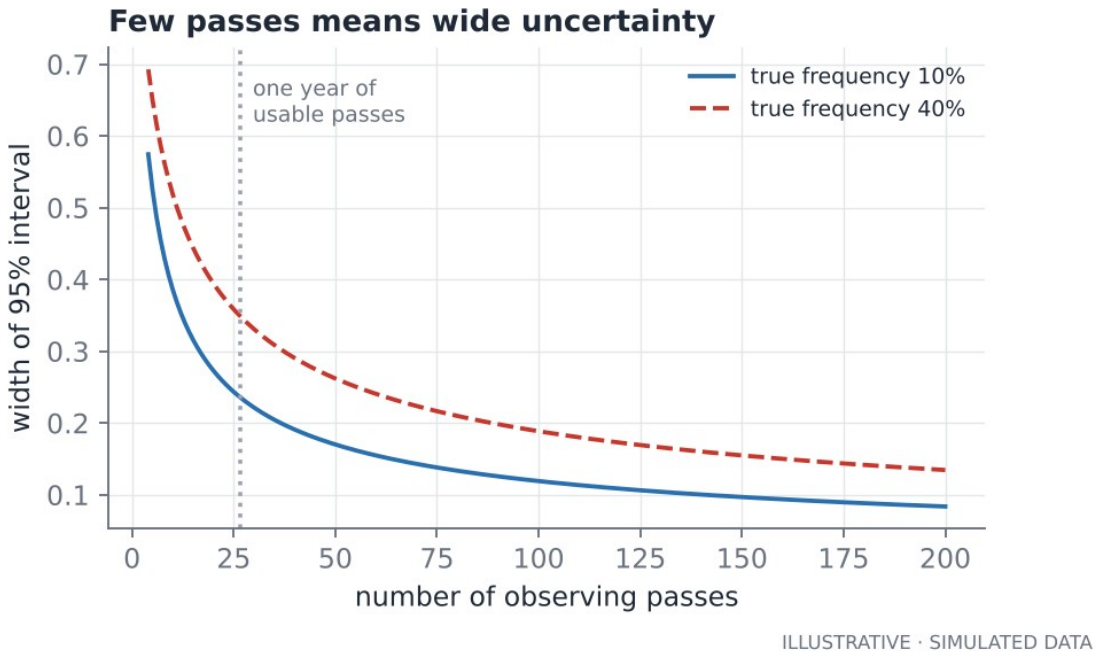


Figure 10.2. The width of the 95 % interval for a detection frequency, against the number of observing passes, for two true frequencies. The dotted line marks one year of usable passes.

The listing is sobering. With about 27 usable passes per year, a true frequency of 0.2 is bracketed only to roughly 0.09–0.38 after a year; with 5 passes the interval spans 0.04–0.62, nearly everything. And **zero events in ten passes is compatible with a frequency as high as 28 %**. Remote-sensing papers that say “no emissions were detected” without saying how many chances there were are making a statement about their sample size, not about the world.

This also shows why a multi-year record matters and why the project's first result on real data should be a **table of observed passes and intervals**, not a map of triumphs.

The control test: is any of this real?

The central question is whether events cluster near known sources more than chance predicts. The **control test** compares two flag rates:

- the **near** rate: flagged bins among bins within 40 km of a source;
- the **far** rate: flagged bins among bins at least 60 km from any source.

Far bins are the control. Any flag there is a false alarm (noise, artefact, an unlisted source). If the near rate is much larger, the method is picking up something related to sources. The ratio of near to far is the **enrichment**. The project gives:

- the two rates and their counts;
- a one-sided Fisher exact test (optimistic, as Chapter 8 explained);
- a **bootstrap over passes** for the ratio's confidence interval.

> **Key idea**
> A near/far enrichment with an honest interval is a stronger statement than any collection of pretty plume maps. If the enrichment interval includes 1, you have no evidence the method detects sources, whatever the maps look like.

The 40 km near and 60 km far values leave a gap of 20 km between them, so bins in the transition zone, which are neither clearly near nor clearly far, are excluded. The gap protects the comparison from plumes that extend slightly beyond the near zone.

Hotspots and what they reveal

Clustering events in space (DBSCAN with a 20 km link) produces **hotspots**. A hotspot with many events on many different days **and no source nearby** is the most interesting output of the whole pipeline: it is either an unlisted source or a persistent artefact, and both are worth knowing. Keep an *unattributed events* table and read it first.

A spatial index you should know: the grid

Besides KD-trees, a regular grid (the project uses 0.1° cells) is the simplest way to composite many sparse passes into a map, by averaging anomalies per cell. Do two things when you draw it: **leave unsampled cells blank** (the gaps are the satellite's coverage, a result in itself), and drop cells with fewer than a minimum number of bins so a single noisy bin cannot colour a cell.

A radial profile: the nearest thing to an average plume

If you average anomaly against distance to the nearest source over all passes, random plume directions cancel the direction but keep the distance dependence. The result is the **radial profile**: it should be positive near sources and drop towards zero with distance. The project builds it with a bootstrap over passes. By construction it is near zero beyond the exclusion radius, because the background was estimated there; the information is in the inner rings.

Key ideas

- Proximity attribution is association, not causation; wind is the next step.
- Complexes are a modelling choice; record the linking distance.
- Keep non-detections so detection frequency has a denominator.
- Few passes mean wide intervals; zero detections is not zero emissions.
- A control test with an honest interval is the project's central evidence.
- Unattributed hotspots are findings.

Exercises

1. Using `s14`, compute how many usable passes per year you would need for the 95 % interval of a frequency near 0.2 to be narrower than 0.15. How many years of OCO-2 data is that?
2. Design a control test for a methane project in which sources are oil wells. What would your near and far zones be, and what could make the far zone *not* a clean control?
3. Two complexes have 4/10 and 40/100 event passes. Compute both Wilson intervals (you may use the project's `wilson_ci`) and explain which result is more informative.

Chapter 11. From enhancement to emission

An enhancement in ppm is not an emission. Policy makers want kilograms per second or tonnes per year. This chapter shows how the first becomes the second, what the answer depends on, and why the uncertainty is large. It also introduces the matched filter, the workhorse for finding methane with imaging spectrometers.

The mass-balance idea

A steady source emits Q (kg/s) into wind of speed U (m/s). In a steady state, the mass of gas crossing any line perpendicular to the wind, downwind of the source, equals Q. Written as a formula:

```
Q  =  U  x  integral over the crosswind distance y of  [ column enhancement (kg/m2) ]  dy
```

Three ingredients: the **column enhancement** in kilograms per square metre (from ppm, using the conversion in Chapter 2), the **wind speed** that carries it, and a **transect** across the plume. This is the *cross-sectional flux* method. Its sister, the **integrated mass enhancement (IME)** method, sums the excess mass in the whole plume and divides by the time the wind takes to flush it out: Q = U_eff × IME / L, where L is a length scale of the plume (often the square root of its area).

A worked example with a known answer

The listing simulates a Gaussian plume with a true emission of 300 kg/s, a 4 m/s wind, and a transect 20 km downwind, with noise representing 8 footprints averaged per row. Then it applies the flux formula.

```
"""s12 - From ppm to kg/s: the cross-sectional flux method, and what it says about detectability."""
import numpy as np

G = 9.80665
M_DRY, M_CO2, M_CH4 = 0.0289652, 0.04401, 0.01604           # kg/mol

def kg_per_m2_per_unit(ps_hpa, molar_mass, unit=1e-6):
    """Mass of gas per m2 of ground for a 1-unit (1 ppm = 1e-6, 1 ppb = 1e-9) column enhancement."""
    return ps_hpa * 100 / (G * M_DRY) * unit * molar_mass

ps = 850.0                                                  # Highveld surface pressure, hPa
k_co2 = kg_per_m2_per_unit(ps, M_CO2)
print(f"1 ppm XCO2 = {k_co2 * 1000:.2f} g CO2 per m2")

def sigma_y(x_m):                                           # crosswind spread of the plume (invented)
    return 0.2 * x_m

def plume_column(y_m, x_m, Q, U):
    """Steady Gaussian plume integrated over height: kg/m2 of ground, at crosswind offset y."""
    s = sigma_y(x_m)
    return Q / (U * np.sqrt(2 * np.pi) * s) * np.exp(-0.5 * (y_m / s) ** 2)

Q_true, U_true, x = 300.0, 4.0, 20_000.0                    # kg/s, m/s, 20 km downwind
y = np.arange(-20_000, 20_001, 2_250.0)                     # one sample per footprint row, 2.25 km
col = plume_column(y, x, Q_true, U_true)
print(f"\ntrue Q = {Q_true:.0f} kg/s = {Q_true * 3.156e7 / 1e9:.1f} Mt CO2/yr; peak enhancement at 20 km:
"
      f"{col.max() / k_co2:.2f} ppm")

def flux_estimate(column_kg_m2, U, dy):                     # Q = U * integral of column across the
plume
    return U * np.sum(column_kg_m2) * dy
```

```
print(f"noise-free, right wind : Q = {flux_estimate(col, U_true, 2250.0):.0f} kg/s")
print(f"wind 30% too high      : Q = {flux_estimate(col, 1.3 * U_true, 2250.0):.0f} kg/s")

rng = np.random.default_rng(8)
ests = []
for _ in range(2000):
    noisy = col + rng.normal(0, 0.8 / np.sqrt(8), y.size) * k_co2   # 8 footprints averaged per row
    ests.append(flux_estimate(noisy, U_true, 2250.0))
print(f"with 0.28 ppm row noise : Q = {np.mean(ests):.0f} +/- {np.std(ests):.0f} kg/s (1 sigma, one
transect)")

# Detectability: what emission gives a 1 ppm peak at 20 km?
Q_min = 1.0 * k_co2 * U_true * np.sqrt(2 * np.pi) * sigma_y(x)
print(f"\nemission that gives a 1 ppm peak at 20 km: {Q_min:.0f} kg/s = {Q_min * 3.156e7 / 1e9:.0f} Mt
CO2/yr")

# The same arithmetic for methane in parts per billion
k_ch4 = kg_per_m2_per_unit(ps, M_CH4, unit=1e-9)
for tph in (10, 50):
    Q = tph * 1000 / 3600
    peak_ppb = plume_column(0, x, Q, U_true) / k_ch4
    print(f"methane source {tph:>2} t/h ({Q:5.2f} kg/s): peak {peak_ppb:5.1f} ppb at 20 km downwind")
```

Output

```
1 ppm XCO2 = 13.17 g CO2 per m2

true Q = 300 kg/s = 9.5 Mt CO2/yr; peak enhancement at 20 km: 0.57 ppm
noise-free, right wind : Q = 300 kg/s
wind 30% too high      : Q = 390 kg/s
with 0.28 ppm row noise : Q = 303 +/- 145 kg/s (1 sigma, one transect)

emission that gives a 1 ppm peak at 20 km: 528 kg/s = 17 Mt CO2/yr
methane source 10 t/h ( 2.78 kg/s): peak  14.4 ppb at 20 km downwind
methane source 50 t/h (13.89 kg/s): peak  72.1 ppb at 20 km downwind
```

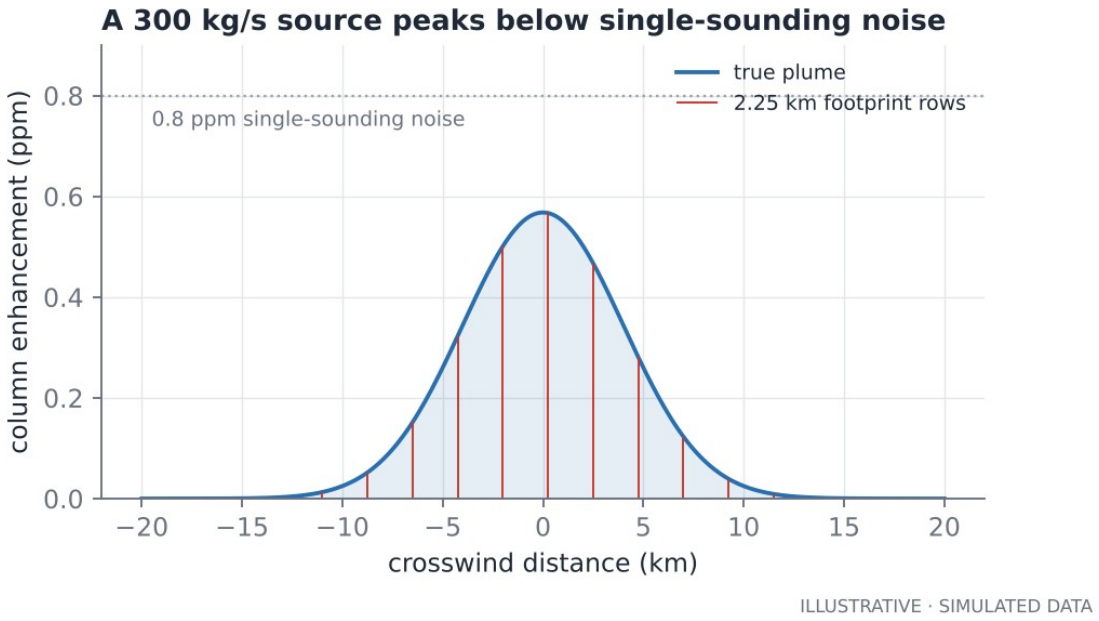


Figure 11.1. A 300 kg/s source seen 20 km downwind by a swath that takes one row every 2.25 km. The peak enhancement is below the single-sounding noise level.

Four lessons in the output

1. **The conversion factor is the glue.** 1 ppm of XCO_2 at 850 hPa is 13.2 g/m².
2. **A 300 kg/s source (9.5 Mt CO_2 per year, a big power plant) peaks at only 0.57 ppm** 20 km downwind in this model. That is below the project's 1 ppm detection floor, and below the 0.8 ppm per-sounding noise. The plume is real, but it would not be flagged.
3. **The emission that gives a 1 ppm peak at 20 km is about 528 kg/s, or 17 Mt CO_2 per year.** In this simple model only the very largest plants could be detected at a 1 ppm floor. Real analyses of OCO-2 power-plant plumes in the literature find that a few of the largest plants are detectable on good overpasses. That is consistent with the sparse results you should expect from the Highveld project: do not be surprised if few events are real.
4. **Wind error is first-order.** A 30 % too-high wind gives a 30 % too-high emission, directly and with no averaging benefit. Reanalysis winds (ERA5, MERRA-2) at the plume height, plus an uncertainty from the spread among products, are mandatory for any quantification. With noise only, one transect gives 303 ± 145 kg/s (1σ): about 50 % scatter before wind error.

> **Treat the model with suspicion**
>
> The plume shape (spread proportional to distance), the steady wind, and the simple geometry are illustrative. Real plumes meander, shear with height, depend on boundary-layer stability and may be wider or narrower. The numbers show orders of magnitude and sensitivities, not a calibrated detection limit for any real power station. For real work, use a proper transport model or a validated retrieval method, and check the literature.

Methane in the same framework

Methane works the same way, with different numbers. Using the same function with the molar mass of CH_4 and ppb units, a 10 t/h methane source gives a peak column enhancement of about 14 ppb at 20 km in this model, and a 50 t/h source about 72 ppb. TROPOMI's noise on a single pixel is of the order of ten ppb or more, so only large sources can stand out from single pixels, and averaging over time and space is used for smaller ones. The literature reports detection thresholds for point sources of several to tens of tonnes per hour depending on conditions; check the specific papers for the instrument and the surface you use. Methane is therefore a *low-contrast, high-volume* detection problem, just like CO_2.

Detecting with spectra: the matched filter

Imaging spectrometers (AVIRIS-NG, EMIT, GHGSat-type instruments) record hundreds of narrow bands for every pixel at tens of metres resolution. For those instruments the enhancement is not retrieved from three bands but found by asking: **in which pixels does the spectrum look like the background plus a little bit of methane absorption?**

The **matched filter** answers that. Let x be a pixel's spectrum, μ the mean background spectrum, Σ the background covariance (how bands vary together over the scene), and t the *target signature*: the change in the spectrum for one unit of gas, computed from the gas's absorption spectrum multiplied by the mean spectrum. The filter's estimate of the gas amount is

```
alpha(x) = (x - mu)^T  Sigma^-1  t  /  ( t^T  Sigma^-1  t )
```

The inverse covariance “whitens” the data: it down-weights bands where the background naturally varies a lot (so surface variations do not look like gas) and up-weights bands where the background is quiet but the gas absorbs. It is the linear detector with the best signal-to-noise for a known signature on Gaussian noise.

```
"""s13 - The matched filter: how imaging spectrometers (AVIRIS-NG, EMIT, GHGSat-style) find methane
plumes."""
```

```
import numpy as np

rng = np.random.default_rng(7)
N, B = 80, 60                                          # 80 x 80 pixels, 60 spectral bands
wl = np.linspace(2100, 2450, B)                         # nm, the SWIR region where CH4 absorbs

# 1) a unit-absorption target signature t(wl): the effect of adding 'one unit' of gas on each band
target = -(0.5 * np.exp(-0.5 * ((wl - 2300) / 12) ** 2) + 0.3 * np.exp(-0.5 * ((wl - 2370) / 15) ** 2))

# 2) a background scene: a few surface types, each with its own spectrum, plus sensor noise
base = np.array([0.20 + 0.0004 * (wl - 2100), 0.12 + 0.0001 * (wl - 2100), 0.30 - 0.0003 * (wl - 2100)])
surf = rng.integers(0, 3, (N, N))
bright = rng.normal(1.0, 0.15, (N, N))                  # albedo varies pixel to pixel
cube = base[surf] * bright[..., None] * (1 + rng.normal(0, 0.01, (N, N, B)))   # (N, N, B)

# 3) plant a plume: gas amount alpha (arbitrary units) is a Gaussian blob
yy, xx = np.mgrid[0:N, 0:N]
alpha_true = 0.05 * np.exp(-0.5 * (((xx - 45) / 12) ** 2 + ((yy - 40) / 5) ** 2))
plumed = cube * (1 + alpha_true[..., None] * target)    # multiplicative (Beer-Lambert, small alpha)

# 4) the matched filter: alpha = (x - mu) . S^-1 t / (t . S^-1 t), with t the target scaled by the mean
spectrum
def matched_filter(cube, target):
    X = cube.reshape(-1, cube.shape[2])
    mu = X.mean(axis=0)
    cov = np.cov((X - mu).T) + 1e-9 * np.eye(X.shape[1])         # background covariance
    w = np.linalg.solve(cov, target * mu)                          # whitened target (scaled by mean)
    alpha = ((X - mu) @ w) / ((target * mu) @ w)
    return alpha.reshape(cube.shape[:2])

alpha_hat = matched_filter(plumed, target)
alpha_none = matched_filter(cube, target)
inside = alpha_true > 0.02
print(f"true alpha in plume core : {alpha_true.max():.3f}")
print(f"estimated, plume core    : {alpha_hat[alpha_true > 0.04].mean():.3f}")
print(f"estimated, no plume scene: std {alpha_none.std():.4f} (noise floor)")
print(f"signal-to-noise of the core: {alpha_hat[alpha_true > 0.04].mean() / alpha_none.std():.1f}")
print(f"pixels above 4 sigma: {(alpha_hat > 4 * alpha_none.std()).sum()} (true plume pixels > 0.02:
{inside.sum()})")
```

Output

```
true alpha in plume core : 0.050
estimated, plume core    : 0.041
estimated, no plume scene: std 0.0111 (noise floor)
signal-to-noise of the core: 3.7
pixels above 4 sigma: 68 (true plume pixels > 0.02: 345)
```

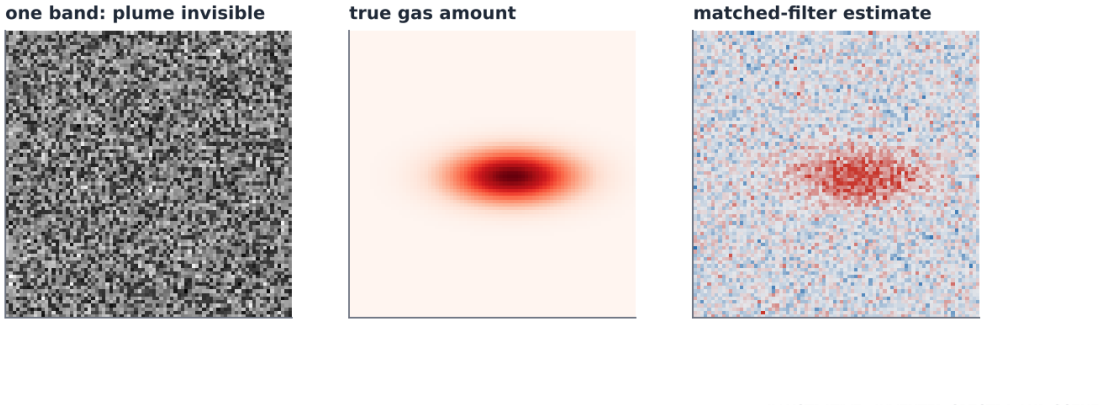


Figure 11.2. In a single band the plume is invisible (left). The matched filter recovers the plume shape (right) from 60 bands of invented spectra.

The listing recovers the plume core with a signal-to-noise ratio of about 3.7, but a little low: 0.041 against a true core mean of 0.045 (about 8 % under). Two honest observations. First, **the bias is not a bug**: the background here is three different surface types mixed together, so one global mean and covariance fits none of them well. Real workflows apply the filter within clusters of similar surfaces, which tightens the covariance, and then check for surfaces whose spectra resemble the gas signature (the main source of false positives). Second, a signal-to-noise of 3.7 is borderline; real detection thresholds pair the filter with a spatial-coherence test (a plume is a connected, plume-shaped blob, not isolated pixels) and then an IME estimate on the connected region.

Inversion: the formal route

When you need regional or national emissions, you move from single plumes to **inverse modelling**: an atmospheric transport model maps emissions to concentrations at observation points, and a Bayesian optimisation finds the emissions that best explain the observations while staying close to a prior estimate, weighted by their uncertainties. In one line:

```
x_hat = x_prior + G (y - K x_prior)      G = S_a K^T (K S_a K^T + S_e)^-1
```

Here x are emissions, y observations, K the transport model's sensitivity, S_a the prior uncertainty and S_e the observation uncertainty. Inversions are the standard way national inventories are cross-checked against atmospheric data. This book does not implement one, but you should recognise the pieces: every uncertainty you estimated in Chapter 8 becomes S_e, and that is why those estimates must be honest.

Reporting emissions

- Report **rates with uncertainty** and the assumptions (wind source, plume model).
- Distinguish **detections** (a plume seen) from **quantifications** (a rate estimated) and from **annual totals** (which need a persistence or intermittency model; a plume seen on a clear afternoon does not mean continuous emission).
- Compare with **bottom-up inventories** (reported activity data × emission factors) only after checking that both refer to the same source and period.

Key ideas

- Q = U × ∫ column enhancement across the plume; wind uncertainty enters one-for-one.
- Know the conversion from ppm (or ppb) to kg/m² for your surface pressure.
- Sensitivity matters: your detection floor in ppm sets a minimum detectable emission rate; compute it before you promise results.

- The matched filter is the standard detector for spectral imagers; bias and false positives come from the background model.
- Inversions combine a transport model, a prior and honest uncertainties.

Exercises

1. In `s12`, change the wind to 2 m/s and 8 m/s. How do the peak enhancement and the minimum detectable emission change? What does that tell you about when to expect detections?
2. In `s13`, apply the matched filter separately to each of the three surface types (use `surf`). Does the bias shrink? Does the noise floor?
3. Suppose a source emits 50 % of the time at 100 kg/s. What annual total would a single clear-sky overpass seeing it “on” suggest if you assumed continuous emission? How would you improve the estimate?

Chapter 12. Designing a remote-sensing project

Good projects are decided before any code. This chapter is the process: how to go from a vague interest to a repository that an expert will trust. Part V applies it to specific topics.

Start with the question, not the dataset

A good question has four parts: a **quantity** (XCO_2 anomaly, flooded area), a **place**, a **time period**, and a **decision it could inform**. “Do coal power plants on the Highveld produce detectable OCO-2 enhancements, and how often?” has all four. “Analyse some CO_2 data” has none.

Then write down, in advance:

1. **What would a convincing answer look like?** (A table of detection frequencies with intervals and a control test.)
2. **What would convince you the answer is* no*?** (An enrichment interval that includes 1.)
3. **What could fool you?** (Artefacts, clear-sky bias, correlated noise, unlisted sources.)

Writing these first stops you from shaping the analysis to fit the result.

The ten stages of a data pipeline

Every project in this field follows the same skeleton. The Highveld project is a worked example of each stage.

Stage	Question	In the Highveld project
1. Acquire	How do I get the data reliably?	`download.py` with version pinning and a file-count guard
2. Inspect	What is really inside?	`inspect_file.py`
3. Read	How do I turn files into a clean table?	`read_oco2.py`
4. Quality control	What do I trust, and how much did I remove?	`qc.py` with a report
5. Model the signal	What is background, what is anomaly?	`anomaly.py`
6. Detect	What rule defines an event?	`events.py`
7. Attribute and summarise	What do events mean, and how often?	`attribute.py`, `composite.py`
8. Validate	How do I know it is not noise?	the control test, simulation tests
9. Persist	Where do results live?	`db.py`, PostGIS schema
10. Communicate	How do others see it?	figures, map, README, notebook

Build in the right order

The order the Highveld tutorial uses (Levels 0 to 12) is not arbitrary and is worth copying: **skeleton and configuration; geography and reference data; data access and inspection; reading and cleaning; a synthetic test bench; analysis; statistics; figures; database; orchestration; tests; documentation.**

The most important choice in that list is the **synthetic test bench built before the analysis**. On real data you do not know the right answer, so a bug in your background estimate looks like a result. A generator whose sources, noise, clouds and artefacts you control lets you ask “does my analysis recover what I put in?” It also lets you ask the question you must answer before you trust anything: *what does my method do when there is nothing there?*

Make the simulation hard

A test bench that is too clean teaches you nothing. The first version of the project's simulator gave a near-perfect enrichment of 383× and almost no false alarms, because it had white noise only. Adding spatially correlated noise, localised artefacts, urban domes and patchy cloud dropped the enrichment to about 10×, a realistic picture. Rule of thumb: **if your method looks flawless on your simulation, your simulation is too easy.**

Repository layout

```
project/
  README.md            status, quick start, method, limitations
  requirements.txt     dependencies with lower bounds
  Makefile             setup, test, demo, real, db-up, clean
  .env.example         names of secrets, never values
  .gitignore           data, secrets, synthetic outputs
  project_name/        the package: one module per stage
  tests/               fast tests that run in seconds
  notebooks/           the guided walkthrough, executed
  sql/                 schema and example queries
  scripts/             one-off builders (reference data)
  data/reference/      small curated inputs, committed
  data/raw/            downloads, NOT committed
  results/             outputs of a real run, committed
  docs/                the long explanations
```

Keep **code**, **raw data**, **reference data** and **results** in separate folders, so you can always tell what is original and what you derived, and so a stray run never overwrites an input.

Configuration, not constants in code

Every tunable choice goes in one frozen configuration object: exclusion radius, bin length, thresholds, minimum counts. Benefits: a run is described by its parameters; sensitivity analysis is a one-line change; and a reader can see all your decisions in one file. The project writes the exact parameters into `summary.json` next to the results, so any output can be traced to the settings that produced it.

```
@dataclass(frozen=True)
class Params:
    """Analysis parameters. Frozen so a run can be reproduced from its Params."""

    # --- quality control ---------------------------------------------------
    keep_modes: tuple = (0, 1)        # operation_mode: 0 nadir, 1 glint (2 target, 3 transition dropped)

    # --- segmenting soundings into overpasses ------------------------------
    pass_gap_s: float = 300.0         # time gap that starts a new pass (a pass over the box lasts ~1 min)
    min_soundings: int = 150          # good soundings needed for a usable pass
```

```
    min_far_soundings: int = 60       # far-from-source soundings needed to define a background
    min_extent_km: float = 100.0      # along-track length needed (short clips cannot define a background)

    # --- background and anomaly -------------------------------------------
```

Provenance: every output records how it was made

For each run save: software version (git commit), parameter values, input file list and date range, quality-control report, the noise estimates, and the random seeds. Six months later you will not remember which threshold you used. A run you cannot reproduce is an anecdote.

Honesty features built into the code

These are cheap to add and make the repository far more credible.

- **Synthetic data carry a flag in the file attributes.** The pipeline prints a banner, stamps figures `SYNTHETIC DATA`, and **refuses to mix** synthetic and real files.
- **The README has a status table** listing what is verified and how, and what has not yet been run.
- **A results placeholder** that says “not yet run on real data” until it is true.
- **A limitations section** that comes before results, not after.

Scoping and time

Be ruthless. A portfolio project that does five things well beats one that does fifteen things half-way. A good scope for a first project: one instrument, one region, one year, one question, one robust control. Put the exciting extensions (wind, a second sensor, machine learning) in a *next steps* list. Estimate your effort in three layers: getting data and reading it (it always takes longer than expected), the analysis, and the validation and documentation (which is half the work).

Key ideas

- Write the question, the convincing answer, the disproving answer and the ways you could be fooled before coding.
- Follow the ten-stage pipeline and build in the proven order; build the synthetic test bench first and make it hard.
- Separate code, raw data, reference data and results.
- Put every tunable parameter in one frozen configuration, and write it with each result.
- Build honesty into the code: flags, banners, refusals, status tables.

Exercises

1. Pick a topic you care about. Write the four-part question and the three “write down in advance” answers.
2. Draw the ten-stage pipeline for your topic. For each stage, name the file you would write and the check you would run.
3. Design a synthetic test bench for a flood-mapping project. What must it include so that it is *not* too easy?

Chapter 13. Databases for geospatial results

Why put results in a database at all?

Files are good for arrays and for one-off analysis. A database is better for **results that people will query**: “which events fall within 40 km of this point?”, “which complexes were observed fewer than 10 times?”, “show detections by month”. A database also forces you to design a schema (which means thinking about what your entities and relationships are), and it is a skill many remote-sensing job descriptions ask for explicitly.

Rule of thumb for choosing storage:

You have	Use	Because
Big arrays (images, cubes)	NetCDF, GeoTIFF, Zarr	Built for arrays and chunked reads
A table you analyse in Python	Parquet or CSV	Simple, portable
Results with geometry that many people will query spatially	PostgreSQL + PostGIS	Spatial indexes, SQL, concurrent access
A quick local analytical database	DuckDB (with its spatial extension)	Needs no server

Relational thinking in five minutes

A relational database stores **tables** of rows. Each table has a **primary key** that identifies a row. A row in one table can point to a row in another with a **foreign key**, which the database enforces. Designing a schema is deciding what the tables are and how they point to each other.

The Highveld schema has six tables, in dependency order:

- `source_complexes`: one row per group of nearby sources (parent);
- `sources`: one row per facility, each pointing to its complex;
- `passes`: one row per overpass, with background and noise estimates;
- `bins`: one row per 5 km bin, pointing to its pass;
- `events`: one row per event, pointing to its pass and to the nearest source;
- `complex_stats`: one row per complex with observed passes, event passes and the interval.

Tables with foreign keys must be **created and loaded after the tables they reference**, and truncated with `CASCADE`. That ordering is why the loader writes parents first.

PostGIS in a nutshell

PostGIS adds **geometry types** and spatial functions. The two things to learn first:

- **`geometry(Point, 4326)`**: a point in longitude/latitude, with spatial reference system 4326 recorded in the column.
- **A GiST index** on that column, which makes “within this distance” or “intersects this polygon” fast. Without it, every spatial query scans the whole table.

The second thing to learn is the difference between **geometry** and **geography**. Casting a column to `geography` makes distance functions return true metres on the ellipsoid, which is what you want for “within 40 km” queries on lon/lat data. Doing the same on geometry in EPSG:4326 returns degrees, which is meaningless as a distance.

The queries that matter

These are the project's own example queries, run against a real PostGIS server during development. Read them as templates.

```
    "complex_frequency": """
        SELECT c.name, s.observed_passes, s.event_passes,
               ROUND(s.frequency::numeric, 3) AS frequency,
               ROUND(s.ci_low::numeric, 3) AS ci_low, ROUND(s.ci_high::numeric, 3) AS ci_high
        FROM complex_stats s JOIN source_complexes c USING (complex_id)
        ORDER BY s.observed_passes DESC;""",
    # events within :km of a point (geography => true metres on the ellipsoid)
```

```
    "events_near_point": """
        SELECT event_id, obs_date, ROUND(peak_anomaly_ppm::numeric, 2) AS peak_ppm,
               ROUND((ST_Distance(geom::geography,
                      ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography) / 1000)::numeric, 1) AS km
        FROM events
        WHERE ST_DWithin(geom::geography, ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography, :km *
1000)
        ORDER BY km;""",
```

The first joins two tables and orders by sampling, so best-observed complexes come first. The second filters events by true distance from a point. Look at three details: `ST_DWithin(..., :km * 1000)` uses the geography type and metres; `:lon`, `:lat` and `:km` are **bind parameters**, never string-pasted values (which would be an injection risk and a source of quoting bugs); and `ST_Distance` computes the distance for display in the same units.

Loading data safely

Three loading lessons came out of this project.

1. Load idempotently. The loader truncates every table and then inserts. Running it twice gives the same result, not duplicated rows. Make any loader safe to rerun.

2. NaN is not NULL. pandas writes a floating-point NaN as the value `'NaN'`, not as NULL. That silently breaks `AVG()`, `COUNT(column)` and every `IS NULL` test. The project's `_null_safe` helper converts NaN to Python `None` first; it also turns numpy integers into plain Python integers, which the database driver can adapt.

```
def _null_safe(df: pd.DataFrame) -> pd.DataFrame:
    """NaN -> None for every non-geometry column.

    Without this, NaN floats are written as the value 'NaN' (not NULL), which silently breaks
    AVG(), COUNT(col) and any 'IS NULL' test. astype(object) also turns numpy ints/bools into
    plain Python values, which the database driver can adapt.
    """
    out = df.copy()
    for c in out.columns:
        if c == "geom":
            continue
        out[c] = out[c].astype(object).where(out[c].notna(), None)
    return out
```

3. Check what is really in the database. After loading, run a query that counts NULLs and `'NaN'` values, compares row counts with the source tables, and reproduces one number from Python in SQL. The project's regression test runs the control test in both Python and SQL and asserts that they agree: this once caught a SQL query with a hard-coded 30 km radius while the Python code used 40.

Indexes and performance, briefly

- Index the columns you filter or join on. Spatial columns get **GiST**; ordinary columns get B-tree (the default).
- A **partial index**, such as one on flagged bins only, can be tiny and fast when the rows you query are a small subset.
- Use `EXPLAIN ANALYZE` to see whether a query used the index. If you see “Seq Scan” on a big table for a spatial filter, the index is missing or not being used.

Running it locally

The repository includes a `docker-compose.yml` that starts a PostGIS 16 container in one command, with a named volume so data persist. For a portfolio this matters: **a reviewer can run your database in one command**, which is far more convincing than a screenshot.

```
docker compose up -d
python -m highveld_co2.pipeline --raw data/raw/synthetic --out results_demo --db
docker compose exec db psql -U co2 -d co2 -c "SELECT COUNT(*) FROM events;"
```

Key ideas

- Use a database for results that people will query spatially; use files for arrays and one-off work.
- Design tables with primary and foreign keys, and load parents first.
- Geometry columns need a GiST index; cast to geography for distances in metres.
- Use bind parameters; convert NaN to NULL; make loaders idempotent.
- Reproduce one number in SQL and Python as a regression test.

Exercises

1. Write the SQL to list the five complexes with the fewest observed passes, together with the Wilson interval columns, from `complex_stats`.
2. Write a PostGIS query that returns every source within 30 km of Pretoria (28.19°E, 25.75°S), using geography and bind parameters.
3. Design a table for **facility-scale methane detections** (plume id, scene, time, emission estimate with uncertainty, facility, wind source). What are its keys, and which columns would you index?

Chapter 14. Visualising honestly

A figure is a claim. Colour, scale, what you leave blank and what you stamp on the corner all change what a reader believes. This chapter collects the rules the project's figures follow, and the reasons behind them.

Choose the form by the job of the data

Ask what the data's job is, then pick the chart:

Job	Form	Notes
Show a magnitude over space	Map with a **sequential** colour scale	One hue, light to dark
Show a signed anomaly (above/below normal)	Map with a **diverging** scale and a neutral midpoint	Zero must look like nothing
Compare a few categories	Bars or dots, direct labels	Start bars at zero
Show change over time	Lines, thin, with light gridlines	Direct labels or a legend if more than one
Show uncertainty	Intervals or bands, never bare numbers	Say what the interval is
Show where you did not look	Leave cells blank	The gaps are results

Colour is data encoding

Three rules cover most mistakes.

1. **Sequential for magnitude, diverging for polarity, categorical only for identity.** Never use a rainbow for numbers.
2. **A diverging scale has two hues and a neutral grey midpoint, centred on zero.** Blue to red, with grey at 0, is the standard for anomalies.
3. **Never rely on colour alone.** Roughly 8 % of men have some colour-vision difference. Use shape, line style or direct labels in addition to hue, and keep a legend whenever there are two or more series.

```
"""s15 - Colour is data encoding: the same anomaly field drawn with a rainbow and with a diverging
scale."""
import numpy as np
from matplotlib import colormaps
from matplotlib.colors import LinearSegmentedColormap
from skimage.color import rgb2lab

diverging = LinearSegmentedColormap.from_list("div", ["#2a6fb0", "#e5e7eb", "#c8372d"])
rainbow = colormaps["jet"]

def lightness(cmap, n=9):
    """CIELAB L* (0 black .. 100 white) along the colour map: how bright each step looks."""
    rgb = cmap(np.linspace(0, 1, n))[:, :3][None, :, :]
    return rgb2lab(rgb)[0, :, 0]
```

```
print("perceived lightness L* along each colour map (left = most negative, right = most positive)")
print("rainbow  :", " ".join(f"{v:4.0f}" for v in lightness(rainbow)))
print("diverging:", " ".join(f"{v:4.0f}" for v in lightness(diverging)))
print("the rainbow gets brighter, then darker: equal data steps do not look like equal steps;")
print("the diverging map is brightest exactly at zero, so 'nothing' looks like nothing.")

# an anomaly field: weak noise with one real plume (+2 ppm) and one weaker low (-1 ppm)
rng = np.random.default_rng(0)
yy, xx = np.mgrid[0:60, 0:60]
field = 0.4 * rng.normal(size=(60, 60))
field += 2.0 * np.exp(-0.5 * (((xx - 38) / 7) ** 2 + ((yy - 25) / 4) ** 2))
field -= 1.0 * np.exp(-0.5 * (((xx - 15) / 7) ** 2 + ((yy - 42) / 6) ** 2))
print(f"\nfield range: {field.min():.1f} to {field.max():.1f} ppm; noise sd 0.4 ppm")
```

Output

```
perceived lightness L* along each colour map (left = most negative, right = most positive)
rainbow  :   13   32   55   90   90   95   71   54   25
diverging:   46   57   69   80   91   79   66   55   46
the rainbow gets brighter, then darker: equal data steps do not look like equal steps;
the diverging map is brightest exactly at zero, so 'nothing' looks like nothing.

field range: -1.8 to 2.5 ppm; noise sd 0.4 ppm
```

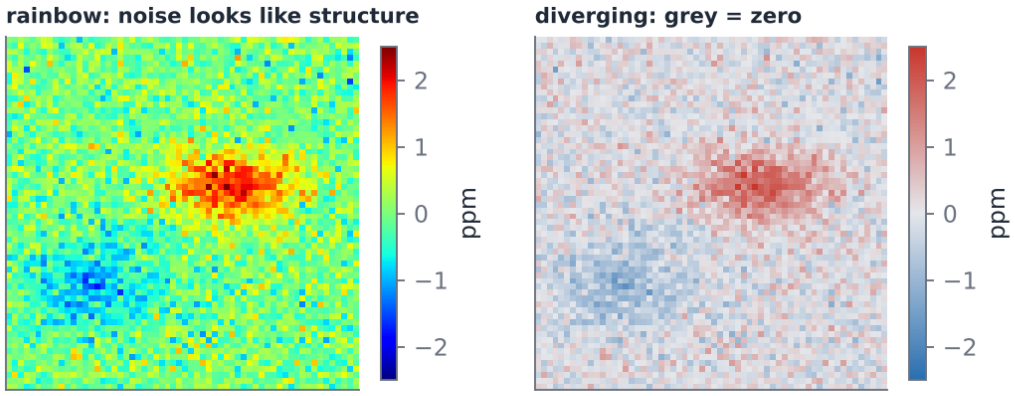


Figure 14.1. The same field in a rainbow scale (left) and a diverging scale (right). The rainbow turns noise into apparent structure and makes the weaker negative anomaly look as important as the plume.

The output explains why. Along the rainbow the perceived lightness (L*) rises from 13 to 95 and then falls back to 25, so equal data steps do not look like equal steps, and bright bands appear at arbitrary places that are artefacts of the colour map. Along the diverging scale lightness peaks at 91 exactly at zero, so “nothing” looks like nothing and sign is obvious.

Marks, grids and text

- **Thin marks.** Lines about 1.5 pt, small markers. Heavy marks hide the data.
- **A quiet grid** (light grey) and no top or right frame.
- **Text in ink colours**, not in the series colour. Colour belongs to the marks; labels stay dark grey or black.
- **Direct labels** where possible (a label next to the line), with no number on every point.
- **One y-axis.** Never put two different scales on one chart; two quantities with different units get two panels.

Maps

- **Show the coordinate system.** Say what projection the map uses, and keep geographic proportions (aspect ratio) sensible.
- **Overlay known sources** so a reader can judge proximity.
- **Leave unsampled cells blank**, and show a note.
- **Mind the stretch.** Symmetric limits for anomalies (−2.5 to +2.5), so that red and blue are comparable.
- **Interactive maps** (Folium, Leaflet) are good for exploration: sources, events and hotspots as toggleable layers. They are not a substitute for a static figure that survives printing.

Uncertainty belongs on the figure

Three cheap ways: intervals on every rate (Wilson), bands around profiles (bootstrap over passes), and showing the **number of observations** per cell or per source. A detection frequency shown as a bare bar is much more persuasive than it deserves to be; the same bar with a wide interval and “n = 5” reads correctly.

Label what is synthetic

Every figure produced from simulated data should carry a visible stamp (the project uses `SYNTHETIC DATA`). All figures in this book carry one for the same reason. Stamped figures cannot be misused by being cropped and pasted into a talk.

Look at every figure

No test catches a label that collides with a title, an axis that clips data, or a legend that hides the peak. The project's control chart once had a value label overlapping its title; it was only found by opening the image. The process is: **generate, open, look, fix, regenerate.** Put it in the task list.

A short checklist

- [ ] Is the form right for the job of the data?
- [ ] Is the colour scale sequential, diverging or categorical for the right reason?
- [ ] Is zero meaningful, and does the scale show it?
- [ ] Is every quantity labelled with units?
- [ ] Is uncertainty or observation count visible?
- [ ] Is synthetic or provisional data stamped?
- [ ] Would it still read correctly in greyscale?
- [ ] Have I opened the image and looked?

Key ideas

- Pick the form by the job; diverging scales for anomalies, with grey at zero.
- Colour alone is never enough; thin marks, quiet grid, dark text.
- Show uncertainty and sampling; leave gaps blank.
- Stamp synthetic data; look at every figure.

Exercises

1. Redraw the anomaly map of `s15` with a sequential scale (single hue). What information does it lose, and when would that be acceptable?
2. Take a figure from this book and write a one-paragraph critique using the checklist above.

3. Add to the radial-profile figure the number of bins in each distance ring. How does it change how you read the outer rings?

Chapter 15. Testing scientific code, and the discipline of honesty

Why scientific code needs different tests

An ordinary program is wrong if it crashes. Scientific code can run perfectly and be wrong: the output looks plausible and nobody notices. So tests in this field have three jobs that ordinary tests do not: **prove the method recovers a known truth, prove statistical claims are calibrated, and pin down every bug that ever happened.**

Five kinds of test, with examples from the project

1. Unit tests of pure functions. Hand-computed answers for small inputs: Wilson intervals match a calculation done by hand (0 events in 10 passes gives an upper bound of 27.7 %); the 16-day cycle arithmetic produces the right dates; version picking keeps exactly one product version.

2. Known-answer tests. Build data where you know the answer and check the code finds it: a planted 2 ppm plume across four bins becomes exactly one event; a background with a planted gradient is followed by a running median to within 0.3 ppm, while a single median cannot do better than 1 ppm; 30 passes of pure noise produce at most 3 flagged bins.

3. Calibration tests. When there is no effect, a significance test should say “significant” about as often as its level promises. The project's null test of the control comparison runs 20 independent noise-only datasets and allows at most three with $p < 0.01$. It uses 20 seeds, not one, because a single seed can fail by chance even when the code is right; before writing that test the author confirmed with 400 runs that the statistic is calibrated.

4. Integration tests. Run the whole pipeline on a small synthetic set: it should complete, produce the files and numbers expected, be **deterministic** (two runs, identical outputs), and **fail helpfully** when given a bad input (a missing variable produces an error that says to run `inspect_file`; mixing synthetic and real files is refused).

5. Regression tests. Whenever a bug is found, write a test that would have caught it, then fix it. The project's SQL control test once used a hard-coded 30 km radius while the Python used 40; now a test compares the two results.

Test type	Asks	Typical failure it catches
Unit	Is this function arithmetic right?	Off-by-one, wrong units
Known-answer	Does the method find what I planted?	Background bias, broken binning
Calibration	Does the false-alarm rate match its promise?	Over-confident statistics
Integration	Does the pipeline run end to end, reproducibly?	Interface drift, silent nondeterminism
Regression	Is a fixed bug still fixed?	Reintroduced bugs

Tests that depend on randomness

Always seed random generators. For a statistical test, run it over many seeds and assert on a **property** (a rate), allowing a small justified number of exceptions. Never loosen a test until it passes without asking why it failed.

The mistakes that taught the most

Nine real mistakes were found while building the project. Each was found by testing or by looking at output, not by thinking harder.

#	What went wrong	What it teaches
1	Attribution radius (30 km) was smaller than the background exclusion radius (40 km); 29 % of events fell in the gap and were orphaned	Parameters that describe one physical effect must be tied together
2	NaN written to the database as the value 'NaN', not NULL	Check what is really stored
3	The Fisher p-value printed exactly 0.0	A zero p-value is a numerical artefact; clamp and say so
4	The SQL control test hard-coded 30 km while Python used 40	Duplicated logic drifts; add a regression test
5	A pandas option only available from version 2.2	Test on more than one library version
6	A first simulation so clean that the enrichment was 383×	A too-good result is a warning; make the simulation hard
7	A statistical test with one random seed failed by chance	Verify calibration, then test over many seeds
8	A label collided with a title on a chart	Always look at the figure
9	A notebook example returned nothing near its chosen facility	Choose examples from the data and read the output

The pattern across all nine: the problem was found because someone **read the output with suspicion**. Build that habit.

Verified, unverified, and how to say so

A mature repository distinguishes three states for each claim:

1. **Verified**: tested against a known answer or an independent source, with the test in the repository.
2. **Unverified but written in good faith**: code that touches something you could not test (here, the real NASA download). Mark it clearly.
3. **Assumed**: a modelling choice you made, such as the single-linkage link distance. List it.

The project's README opens with a status table that sorts every component into these states. Doing the same in your own work costs ten minutes and buys credibility with every expert who reads it.

The ladder of claims

When you have a result, climb only as high as the evidence lets you.

Rung	Claim	Evidence you need
1	The method runs and recovers planted truth	Simulation tests
2	The method flags more near known sources than far from them	Control test with an honest interval
3	A specific detection is plausible	Visual check, repeatability across days, no artefact explanation

Rung	Claim	Evidence you need
4	A specific source emits an enhancement of X ppm	Repeated detections with intervals and wind
5	A specific source emits Y kg/s	Quantification with a full error budget
6	Annual emissions are Z	Intermittency model, many observations, validation

A first real run of this project should reach rung 2, perhaps rung 3. A report that jumps from rung 1 to rung 5 is an overclaim, however good the figures are.

Code review for science

When reviewing your own work (or anyone's) ask: *what is the unit of replication; where could a unit mismatch hide; what happens with an empty input; what happens when every sounding is cloudy; is every threshold in the configuration; is there a test that fails if this line is wrong?*

Key ideas

- Scientific code needs known-answer, calibration and regression tests, not only unit tests.
- Seed randomness; test properties over many seeds.
- Read output with suspicion; a result that is too good is a warning.
- Label every component verified, unverified or assumed.
- Claim only as high on the ladder as your evidence reaches.

Exercises

1. Pick one function you have written. Write one known-answer test and one test that checks behaviour on empty input.
2. Design a calibration test for a z-score threshold of 2.5: what is the expected false-alarm rate, how many simulated datasets would you run, and what rule would you use to decide pass or fail?
3. List the three claims you would be most tempted to overstate in a first project, and the rung each belongs on.

Chapter 16. Delivering the work

A technically excellent project that nobody can run or understand has little value. This chapter covers documentation, notebooks, publishing, and how to write for decision-makers, such as the audience of a United Nations environment programme.

The README is your front door

A reader decides in about thirty seconds whether to continue. The order that works:

1. **One paragraph** saying what the project does and for which region and instrument.
2. **A status table**: what is verified and how, what is not, and what results exist (none, if none).
3. **Quick start**: commands that work on a clean machine and finish in minutes (the project's synthetic demo takes about ten seconds).
4. **How to run on real data**, step by step, including the account and authorisation steps.
5. **Method in one paragraph**, then a link to the longer document.
6. **Limitations**, before results.
7. **Data and licences**, with citations.
8. **Results**, only after a real run, with the date range and the control-test numbers.

Test the quick start on a clean virtual environment before you publish. The project was verified in a freshly created environment for this reason: the most common repository failure is a missing dependency the author's machine happened to have.

Notebooks that teach

A notebook is for **explaining and exploring**; a package is for **repeating**. Keep real logic in the package and let the notebook import it, so the notebook cannot drift from the code. Rules for a good one:

- Start with the question and end with what you learned.
- **Run it top to bottom and save the outputs**, so readers see results without running anything. A notebook with errors in its saved outputs is worse than none.
- Make cells small and each answer one question, with a sentence before it saying why.
- Include exercises and a “what could be wrong” section.
- Let it detect real data when present and fall back to synthetic data with a loud banner.

Documentation at three levels

Level	Audience	Contents
README	Anyone arriving	What, status, how to run
Overview document	A manager or colleague	The question, the method, what is verified, limitations, next steps
Build tutorial	A learner who wants to rebuild it	Level-by-level: what each file does, why in that order, how to check it

The Highveld project has all three, and the third is also what you can talk through in an interview: you can explain every file and why it exists.

Reproducibility checklist

- [ ] One command installs everything; versions are recorded.

- [ ] One command runs the demo; another runs the real analysis.
- [ ] Parameters live in one place and are saved with each output.
- [ ] Random seeds are fixed.
- [ ] Raw data are downloaded by script, not by hand, and their versions recorded.
- [ ] Tests run in under a minute and are run before every commit.
- [ ] Secrets are in `.env`, listed in `.gitignore`, with an example file committed.
- [ ] A licence file is present, and data licences are stated.

Git habits

Commit small, with messages that say *why*. Never commit raw data, secrets, or synthetic outputs that might be mistaken for results. Tag releases when the status table changes. If you commit a secret by accident, **rotate it immediately**; deleting it from the latest commit does not remove it from history.

Continuous integration

A simple CI workflow that installs dependencies and runs the tests on every push gives you a badge on the README and catches environment drift. The project's tests are designed for it: fast, no network, with the database test gated on an environment variable.

Writing for decision-makers

An international environment body, a ministry or a regulator reads differently from a scientist. Principles:

- **Lead with the finding and its confidence**, then the method. “We detected enhancements associated with two of nine source complexes; for the other seven there were too few observations to say” is a complete, usable sentence.
- **Say what the data cannot tell them.** “No detection” is not “no emission”.
- **Use ranges and plain words** for uncertainty: “between 9 and 38 % of passes”, not “0.20 (+0.18/−0.11)”.
- **Make a figure that stands alone**, with a title that states the point, a stamp if provisional, and a source line.
- **Separate the three levels**: what was seen, what it probably means, what should be done. Only the first is a measurement.

Measurement, reporting and verification

Inventory-based climate reporting follows the idea of **measurement, reporting and verification (MRV)**. National greenhouse-gas inventories are normally **bottom-up**: activity data (tonnes of coal burnt, number of wells) multiplied by emission factors, reported under methodology tiers set out in the IPCC guidelines. Satellite and atmospheric data provide **top-down** estimates that can check them. Verification means comparing the two and explaining the differences. Your role as a technical person is to produce top-down numbers that carry honest uncertainty, so that a gap between the two can be interpreted rather than just argued over.

> **The audience for a methane-monitoring role**
>
> Organisations that monitor methane emissions from space typically combine several steps: detecting plumes with satellite data, attributing them to infrastructure, notifying the responsible parties and governments, and following up. The skills they look for are the ones in this book: a detection method with a measured false-alarm rate, attribution with stated confidence, quantification with an error budget, a database of detections, and clear communication. Check the programme's own public documentation for its current methods and data sources.

Key ideas

- The README is a funnel: purpose, status, quick start, real-data steps, limitations, results.
- Notebooks teach; packages repeat; keep logic in the package.
- Three documents serve three audiences.
- Reproducibility is a checklist, not a feeling.
- Lead with the finding and its confidence; state what the data cannot say.

Exercises

1. Write the status table for a project of your own.
2. Rewrite a result sentence from a past analysis so that it states the finding, its confidence and what it cannot tell the reader.
3. Create a minimal CI workflow file that installs requirements and runs pytest. What would you add to make the database test run there?

Chapter 17. Playbook: methane

Why methane matters, in numbers to check

Methane (CH_4) is the second most important human-caused greenhouse gas after CO_2. Over 20 years a tonne of methane warms the climate roughly 80 times as much as a tonne of CO_2, and over 100 years about 27–30 times (figures from the IPCC Sixth Assessment Report; check the current values when you quote them). Its atmospheric lifetime is about a decade, so cutting emissions slows warming quickly. The main human sources are oil and gas, coal mining, waste (landfills and wastewater) and agriculture (livestock and rice); wetlands are the main natural source. A large share of oil-and-gas and coal emissions comes from a **small number of very large, often intermittent emitters** ("super-emitters"), which is why satellites, which can look at everywhere, are useful for finding them.

Methane versus CO_2 as a detection problem

Methane is, in one important sense, *easier* than CO_2. The background is about 1,900 ppb. A 50 ppb plume is a 2.6 % enhancement. For CO_2 a 1 ppm plume on 415 ppm is 0.24 %. A point source of methane stands out roughly ten times more strongly **relative to its background**, because methane is a trace gas with a small background and big point sources, whereas CO_2 has a large background and emissions are spread across the economy and the biosphere.

The challenges are different:

- **Methane absorption bands** (near 1.65 and 2.3 µm) are more affected by surface type and by water-vapour overlap than CO_2 bands.
- **Natural and diffuse sources** (wetlands, rice, cattle) create broad enhancements that are not point sources.
- **Facility-scale emitters** (a single compressor, a vent) are small (metres to hundreds of metres), below the pixel size of global mappers.

Three classes of instruments

Class	Examples	What it does well
Global mappers	TROPOMI (Sentinel-5P), GOSAT	Daily-to-weekly global coverage at kilometre pixels; finds large emitters and regional patterns; free data
Targeted or area imagers	Hyperspectral (EMIT, PRISMA, EnMAP), commercial (GHGSat, Carbon Mapper's Tanager), aircraft (AVIRIS-NG)	Tens-of-metres pixels; images individual plumes; usually tasked at chosen sites
Broadband multispectral	Sentinel-2, Landsat 8/9	Not designed for methane, but their shortwave bands can reveal very large plumes under good conditions; huge archives

A common operational design uses the global mappers as a **wide net** to find hotspots, then tasks the targeted imagers to **locate and quantify** at facility scale, then validates against aircraft or ground measurements when available. Think of it as the same detection pipeline at two resolutions.

Mapping the Highveld pipeline onto methane

Almost every stage of the CO_2 project has a methane counterpart. If you can build one you can build the other.

CO_2 project stage	Methane version (global mapper)	What changes
Acquire (`download.py`)	TROPOMI Level-2 CH_4 files from the Copernicus Data Space	Different portal, STAC/OData search; daily swaths
Inspect and read	NetCDF with a `qa_value` quality score and CH_4 column in ppb	New variable names; use the product's recommended quality threshold
QC	Remove low-quality pixels, cloudy scenes, snow, very dark or bright surfaces	Different surface rules
Along-track coordinate	Not needed: TROPOMI is an image with a wide swath	Replace with 2-D neighbourhoods
Background	Median in a moving window of 50–100 km, excluding the candidate plume region; or an upwind region	2-D rather than 1-D
Anomaly	Pixel minus background, in ppb	Units
Detect	Threshold in ppb **and** a connectedness test (a plume is a connected blob, not isolated pixels)	Spatial coherence replaces along-track bins
Attribute	Nearest infrastructure within a radius, plus wind direction	Wind is essential because plumes are long relative to pixels
Quantify	IME or cross-sectional flux with reanalysis wind	Chapter 11
Statistics	Detection frequency = detections / clear-sky observations	Same denominator logic
Control	Flag rate near infrastructure versus in regions without it	Same idea
Persist	PostGIS tables with plume polygons and facility links	Polygons, not points

The statistical backbone does not change: background, anomaly, thresholds on both size and significance, frequency with a denominator, an honest control. What changes is the geometry and the units.

Detection and quantification, step by step

1. **Quality-screen** the pixels. Keep only pixels the product marks as good, and drop scenes with extensive cloud or snow cover.
2. **Estimate a local background** with a robust statistic, excluding the suspected plume area, and compute the **enhancement** in ppb.
3. **Find connected regions** where the enhancement exceeds a threshold that you have calibrated against false positives. Require a minimum number of pixels and a plausible shape.
4. **Check for artefacts.** Does the enhancement coincide with a surface feature (dark water, a bright pan, a mining area with changing albedo)? Is it in a region of known retrieval problems? Repeat on adjacent days.
5. **Get the wind** at plume height from a reanalysis, with its spread across products as an uncertainty.
6. **Compute IME and Q** with a plume length scale, with the full uncertainty budget (Chapter 11).

7. **Attribute** to infrastructure by upwind search: is there a facility upwind within a distance consistent with the plume length?
8. **Report** with confidence level and what would raise it (a follow-up observation at higher resolution).

What an expert reader will probe

- **How did you calibrate your false-alarm rate?** Show a control on a region without infrastructure, or on shuffled dates.
- **How did you choose the threshold?** Show sensitivity to it.
- **What did you do about surface artefacts?** Name the surfaces you excluded.
- **What is the wind uncertainty?** Show how the emission changes with a different reanalysis.
- **What is the observation denominator?** Show clear-sky counts per target.
- **Do repeated detections agree?** Persistence at the same site across days counts for more than one beautiful plume.

A first methane project you could build next

A small, honest project that reuses the Highveld skeleton:

1. Choose one region with expected strong methane sources (a coal-mining district or an oil-and-gas basin; search the literature for published hotspots so you have something to compare with).
2. Download a year of TROPOMI Level-2 CH_4 over a 5° box.
3. Reuse the project's structure: reader, QC report, background and anomaly (now 2-D), detector with connectedness, attribution to a facility list, frequency with intervals, a control, PostGIS.
4. Build the synthetic test bench first, with Gaussian plumes of known rate, so you can measure detection probability against emission rate.
5. Compare your hotspots with published ones. Agreement is evidence; disagreement is a finding to explain.
6. Write the status table and ladder of claims honestly.

> **About programmes that act on detections**
>
> International programmes that monitor methane use satellite detections to notify governments and operators about large emission events, and track follow-up. The United Nations Environment Programme's International Methane Emissions Observatory (IMEO) runs one, the Methane Alert and Response System (MARS). For current methods, data sources and scope, read the programme's public documents rather than relying on this paragraph.

Key ideas

- Methane is a stronger signal relative to its background than CO_2, but harder in surface artefacts and diffuse sources.
- Use global mappers as a net and targeted imagers as a microscope.
- The CO_2 pipeline transfers stage by stage; the new work is 2-D geometry, wind and units.
- Always show a control, a threshold sensitivity, the denominator and the wind uncertainty.

Exercises

1. Using the structure of `s12`, compute the peak column enhancement in ppb for a 5 t/h methane source at 10, 20 and 40 km downwind. How does the 1σ TROPOMI noise you would assume limit detection at each distance?

2. Write the status table and the ladder of claims for your proposed methane project.
3. List five ways a TROPOMI “plume” might be an artefact, and one test for each.

Chapter 18. Playbooks: floods, rainfall, drought, fire, land cover and air quality

Each section follows the same template: the question, the data, the method (with the chapter that teaches the tools), how to validate, what goes wrong, and a portfolio project. Treat the sensors as starting points and verify current availability and resolutions before you commit.

Floods

Question. Where and how deep did the water go, who was affected, how long did it last?

Data. Radar for extent through cloud (Sentinel-1), optical for clarity when skies clear (Sentinel-2, Landsat), a digital elevation model to estimate depth and to mask implausible areas, a permanent-water layer to remove rivers and lakes, rainfall (IMERG) for context, population and building layers for exposure.

Method. Before/after radar backscatter change or thresholds; water indices on optical imagery (Chapter 6); remove permanent water; mask high-slope and radar-shadow areas; clean up with connected-component filters; estimate depth by intersecting the extent with the DEM (a first approximation); count exposed people or buildings with a spatial join.

Validate. Against a high-resolution image, an aerial survey or a crowd-sourced inventory; report precision, recall and IoU (Chapter 6) and an area with an interval.

Pitfalls. Urban floods (radar double-bounce raises backscatter, so water can look *brighter*), flooded vegetation, wind-roughened water, shadows, timing relative to the peak (satellites often miss the maximum).

Portfolio project. A flood event map for a recent flood in your region with an honest accuracy assessment and an exposure table in PostGIS.

Rainfall

Question. How much rain fell, when, and how does it compare with normal?

Data. GPM IMERG (about 0.1°, half-hourly, with early/late/final runs that differ in latency and gauge correction), CHIRPS (long record, rainfall blended with gauges), national gauge data for validation.

Method. Accumulate to the time scale you need; compute climatology and anomalies per calendar month (Chapter 7); use SPI for drought; build maps with leave-blank for missing data.

Validate. Against rain gauges: bias, correlation and root-mean-square error, split by season and by rainfall intensity.

Pitfalls. Comparing a point (gauge) with an area (pixel); orographic rainfall; dry-season zeros; changing inputs in the merged product over time; the difference between early and final runs.

Portfolio project. A 20-year rainfall anomaly atlas for a catchment with a gauge validation table.

Drought and vegetation stress

Question. Is vegetation or soil moisture unusually poor, and since when?

Data. NDVI or EVI from MODIS or Sentinel-2, rainfall (SPI), soil moisture (for example SMAP), land-surface temperature.

Method. Compute per-pixel, per-calendar-month baselines; Vegetation Condition Index or standardised anomalies; combine with SPI to separate lack of rain from other stress; mask irrigated and non-vegetated areas.

Validate. Against yield statistics, crop reports or field visits.

Pitfalls. Cloud gaps in the wet season (the season you care about most), phenology shifts (planting earlier or later is not stress), changing land use.

Portfolio project. A monthly drought monitor for a province, using SPI and VCI, with a yield correlation.

Fire

Question. Where is it burning now, what burned, how severely?

Data. Active-fire detections (VIIRS 375 m and MODIS 1 km) for near-real-time; burned-area products (MODIS, Sentinel-2-based) for extent; Sentinel-2 or Landsat for dNBR severity mapping (difference of NBR before and after).

Method. Active fire: use the product's confidence flags and cluster detections into fire events (DBSCAN, Chapter 10). Burned area: compute NBR before and after (Chapter 6), threshold the change, mask clouds, and validate.

Validate. Against agency fire perimeters or high-resolution imagery.

Pitfalls. Satellite overpass timing misses short fires, small fires fall below the pixel, cloud and smoke obscure, and agricultural burning and gas flares appear in fire products.

Portfolio project. A fire-season summary of a protected area, with burned area per month and a persistence analysis.

Land cover and change

Question. What is on the ground, and how did it change?

Data. Sentinel-2 and Landsat time series; existing global products (for example ESA WorldCover and Dynamic World) as comparison; training samples from high-resolution images or field points.

Method. Select features (bands, indices, seasonal statistics, terrain); train a random forest (scikit-learn); predict; post-process; assess accuracy with an independent test sample drawn by a clear design.

Validate. Confusion matrix, per-class precision and recall, and **area-adjusted estimates with confidence intervals** following the good-practice recommendations referenced in Chapter 6.

Pitfalls. Spatial autocorrelation in training and test samples (pixels next to each other are not independent: split by region, not at random), class imbalance, label noise, and extrapolating to places unlike the training area.

Portfolio project. A land-cover change map between two years for a peri-urban area with area-adjusted statistics.

Air quality and combustion (the CO_2 recipe, again)

Question. Where are nitrogen dioxide (NO_2) enhancements from cities, power plants and industry?

Data. TROPOMI NO_2 columns (daily, kilometre scale), with quality values; wind reanalysis; facility and city locations.

Method. NO_2 has a short lifetime (hours), so plumes are compact and close to sources: the Highveld pipeline (background, anomaly, threshold, attribution, frequency, control) applies almost unchanged, with a quality threshold on the product, 2-D neighbourhoods, and averaging over many days under wind rotation to find persistent sources.

Pitfalls. Dependence on cloud and surface brightness, changes in lifetime with season and sunlight, and the column versus surface concentration difference.

Portfolio project. A map of persistent NO_2 sources in a region with a control and detection frequencies.

The universal start-up checklist

Step	Questions to answer
1	What is the quantity, the place, the period and the decision?
2	Which sensor measures something physically linked to it, and what does it assume?
3	What is the sampling (swath, revisit, local time, cloud) and what will the denominator be?
4	What is the background or baseline, and how will I estimate it?
5	What is the threshold, how will I calibrate false alarms, and what is my control?
6	What independent data can validate it?
7	What could fool me, in this region, in this season?
8	What will I put in the status table as verified and unverified?

Key ideas

- Every topic fits a template: question, data, method, validation, pitfalls.
- The same ideas recur: background and anomaly, threshold plus significance, denominator, control, validation.
- Radar for cloud, optical for detail, thermal for fire, microwave for rain.
- Pick a project that has an independent validation dataset.

Exercises

1. Choose one topic above. Fill in the eight-step start-up checklist.
2. For land cover, explain in one paragraph why a random split of pixels into training and test sets overstates accuracy.
3. For floods, list four reasons the satellite might miss the peak water extent.

Chapter 19. A learning plan, and how to talk about your work

What you can already do, and the gaps

You are a strong software engineer with Python, SQL-adjacent data work, testing habits, and the discipline of delivering systems for production. Those are exactly the skills many remote-sensing teams lack. Your gaps are domain knowledge (radiative transfer, atmospheric science), statistical depth with spatial and correlated data, and fluency with the data ecosystem. The plan below fills the gaps in order of payoff.

A twelve-week plan

Week	Do	Produce
1–2	Read Chapters 1–4. Run every listing. Open a real OCO-2 file and a TROPOMI file with `inspect_file`-style code	A one-page “five questions” sheet for each dataset
3	Run the Highveld project on real OCO-2 data (README steps 1–7)	The first real results table, with a status table that says what is verified
4–5	Chapters 5–7. Build the flood map and the SPI monitor from scratch on real data of your region	Two small repositories or notebooks with accuracy numbers
6–7	Chapters 8–10. Measure the false-alarm rate of your detector with a control; vary every threshold	A sensitivity table and a calibrated detector
8	Chapter 11. Add wind from a reanalysis; estimate emissions with an error budget	A quantification notebook with uncertainty
9	Chapters 12–13. Move results into PostGIS; write five spatial queries	A schema, a loader and queries
10	Chapters 14–16. Redo all figures; write README, overview and tutorial	A reviewer-ready repository
11–12	Chapter 17. Build the TROPOMI methane project from the Highveld skeleton	A second repository, with a synthetic test bench and a control

Budget your attention for week 3: the first run on real data will surprise you. The two likeliest surprises are a different variable name or fill value in the real files, and many fewer soundings surviving quality control than you expected. Both are normal.

Skills map: what to be able to demonstrate

Skill	Evidence in your repository	Book chapter
Understand what the satellite measures	A README section on the measurement chain and retrieval assumptions	1–2
Handle real formats and coordinates	Reader with fill values, CRS conversions, tests	4–5
Detect signals in noise	Background, anomaly, thresholds, a measured false-alarm rate	8–9
Do spatial analysis	KD-tree attribution, clustering, grids	10

Skill	Evidence in your repository	Book chapter
Quantify	A flux estimate with an error budget and wind uncertainty	11
Engineer for reproducibility	Config, provenance, Makefile, tests, CI	12, 15, 16
Manage data	PostGIS schema, loaders, queries	13
Communicate	Honest figures, status table, three documents	14, 16

How to talk about this work

Three principles for interviews, reports and your repository's front page.

1. **Be precise about what you did and what you did not.** “I built a complete pipeline and verified it on simulated data with known answers, including a control test and a calibration test; the first run on real OCO-2 data is next” is stronger than “I detected emissions from power plants”, especially if the latter is not yet true.
2. **Tell the story of a mistake.** Experts trust people who can describe how a result fooled them. The 30 km versus 40 km radius, the too-clean simulation and the single-seed test are three good ones, with what you changed because of them.
3. **Connect the method to the decision.** “Detection frequency with an interval tells a regulator how many looks they would need to confirm a source” turns statistics into policy relevance.

A short narrative, in your own words

You can adapt this:

> “I wanted to show I could run an end-to-end remote-sensing analysis. I built a pipeline for OCO-2 column CO_2 over South Africa's coal-power cluster: read the data, estimate a per-overpass background, detect enhancements, attribute them to known plants, and count how often each source is actually observed. Because real plumes are hard to verify, I tested everything on simulated data with a known truth, including a control that measures the false-alarm rate. I stored the results in PostGIS. The repository states clearly what is verified and what is not, and I am now running it on real data. The same structure transfers to methane with different instruments.”

What to build next, in order of value

1. **Run the Highveld project on real data** and fill in the Results section honestly.
2. **A TROPOMI methane hotspot project** (Chapter 17) reproducing known published hotspots.
3. **A flood event map with a validated accuracy** (Chapter 18).
4. **A small web map** (Folium or a static site) showing detections with intervals.

Habits that compound

- Keep a **lab notebook** of surprises: date, what you expected, what happened, what it taught you.
- **Read one paper a week** in the area you are building, and note what its error budget looks like.
- **Reproduce before you extend.** Reproducing a published result is the best training.
- **Share early with someone who will disagree**; criticism of the status table is worth more than praise for the figures.

Key ideas

- Your engineering strengths are rare in this field; invest in physics, statistics and the data ecosystem.
- Follow the twelve-week plan; week 3 (real data) is the real test.
- Describe your work with exactly the confidence your evidence supports.
- Build the next projects by reusing the skeleton.

Exercises

1. Write your own 150-word narrative of your project, including one mistake you found.
2. Fill the skills map with the actual file names in your repository that demonstrate each skill.
3. Choose the next project and write its four-part question and status table before writing any code.

Appendix A. Glossary

Term	Meaning
Absorption band	A wavelength range where a gas absorbs light strongly
Active sensor	A sensor that emits its own signal (radar, lidar) and measures the return
Air mass	The relative length of the light path through the atmosphere compared with a vertical path
Albedo	Fraction of sunlight a surface reflects, over all wavelengths
Anomaly	A value minus an expected (baseline or background) value
A priori	The prior estimate a retrieval leans on where the data carry little information
Attribution	Linking an observed enhancement to a source (association, not proof)
Averaging kernel	How sensitive a retrieved quantity is to the true value at each height
Background	The value expected without the local source
Backscatter	Radar energy returned toward the sensor
Bias	A systematic, non-random error
Bootstrap	Estimating uncertainty by resampling the data many times
CRS	Coordinate reference system: how numbers map to places
Calibration	Converting instrument counts to physical radiance
Complex (source)	A group of sources too close to separate, clustered together
Control (test)	A comparison region where no effect should exist, used to measure false alarms
Column	The total amount of a gas in a vertical path from ground to space
DBSCAN	A density-based clustering algorithm; with one minimum sample it is single-linkage
Detection frequency	Events divided by passes that actually observed the source
Diverging scale	A colour scale with two hues and a neutral midpoint, used for signed values
Effective sample size	The number of independent observations equivalent to a correlated sample
Enhancement	The excess above background
False alarm	A flag where no real source exists
Far field	Soundings far enough from known sources to define the background
Fill value	A code in a data file that means no data
Footprint	The ground area one measurement covers
Ground track	The path of the satellite's sub-point on the Earth
GiST index	A PostgreSQL index type for geometry
IME	Integrated mass enhancement: total excess mass in a plume
Inversion	Estimating emissions from concentrations using a transport model
IoU	Intersection over union: overlap of predicted and true areas
KD-tree	A data structure for fast nearest-neighbour queries

Term	Meaning
L1 / L2 / L3 / L4	Levels of processing from calibrated radiance to derived products
Matched filter	A linear detector optimised for a known signal in correlated noise
MAD	Median absolute deviation, a robust measure of scatter
MRV	Measurement, reporting and verification
NDVI / NDWI / MNDWI / NBR	Normalised-difference indices for vegetation, water and burns
NetCDF / HDF5	Self-describing array file formats
Optical depth	The exponent in Beer–Lambert; proportional to the amount of absorber
Overpass / pass	One crossing of a region by the satellite
Passive sensor	A sensor that measures existing radiation, such as reflected sunlight
Plume	A coherent downwind trail of a gas from a source
ppm / ppb	Parts per million and parts per billion by mole fraction
Precision / accuracy	Repeatability versus closeness to truth
Quality flag	A per-observation code saying how much to trust it
Reflectance	Fraction of incoming light reflected by a surface at a wavelength
Retrieval	The algorithm that infers a quantity from measured radiance
Robust statistics	Statistics insensitive to outliers
SAR	Synthetic aperture radar
Sounding	One retrieval from a sounding instrument such as OCO-2
SPI	Standardised precipitation index
Standard error	The standard deviation of an estimate (sigma over root n for independent data)
Sun-synchronous	An orbit that crosses each latitude at the same local solar time
Swath	The width of ground an instrument sees on one pass
Wilson interval	A confidence interval for a proportion that behaves well at small counts
XCO_2 / XCH_4	Column-averaged dry-air mole fraction of CO_2 or CH_4
z-score	A deviation divided by its standard deviation or standard error

Appendix B. Formula cheat sheet

Light and gas

Quantity	Formula	Note
Beer–Lambert	$I = I_0 \exp(-\tau)$	τ is optical depth
Two-way air mass	$m \approx 1/\cos \theta_s + 1/\cos \theta_v$	Sun and view zenith angles
Column dry air	$N = p_s / (g M_{air})$	p_s in Pa, g 9.80665, M_{air} 0.0289652 kg/mol
XCO_2	$\Sigma (\Delta p_i / p_s) x_i$	Pressure-weighted mean mole fraction
Mass per m^2 per unit mole fraction	$N \times 10^{-6} \times M_{gas}$ (for ppm)	13.2 g CO_2 per ppm at 850 hPa
Normalised difference	$(a - b)/(a + b)$	NDVI, NDWI, MNDWI, NBR

Orbits and sampling

Quantity	Formula	Example
Orbital period	$T = (\text{days} \times 1440) / \text{orbits}$	98.9 min for 233 orbits in 16 days
Track spacing at equator	C / orbits	172 km
E-W spacing at latitude φ	$\text{spacing} \times \cos \varphi$	155 km at 26°
Share of ground covered	swath / spacing	about 6.7 % at 26°S

Statistics

Quantity	Formula	Note
Standard error	$\sigma / \sqrt{n}$	Independent errors only
z-score	mean / SE	Naive when errors are correlated
Effective sample size	$n (1 - \rho)/(1 + \rho)$	AR(1) errors
Spread inflation	$\sqrt{((1 + \rho)/(1 - \rho))}$	Large-n limit
Robust σ	1.4826 × median of abs(x − median x)	MAD-based
Wilson interval	centre = $(p + z^2/2n)/(1 + z^2/n)$; half = $z\sqrt{(p(1-p)/n + z^2/4n^2)}/(1 + z^2/n)$	z = 1.96 for 95 %
Precision / recall / IoU	TP/(TP+FP); TP/(TP+FN); TP/(TP+FP+FN)	Binary maps
Matched filter	$(x - \mu)^T \Sigma^{-1} t / (t^T \Sigma^{-1} t)$	Estimated gas amount

Emissions

Quantity	Formula	Note
Cross-sectional flux	$Q = U \int \Delta\Omega \, dy$	$\Delta\Omega$ in kg/m²
IME method	$Q = U_{eff} \times IME / L$	L plume length scale
Gaussian plume, height-integrated	$\Delta\Omega(y) = Q / (U \sqrt{(2\pi)} \sigma_y) \exp(-y^2 / 2\sigma_y^2)$	Illustrative only
Annual mass from a rate	kg/s × 3.156×10^7 s	1 kg/s = 0.0316 Mt/yr
Bayesian inversion	$\hat{x} = x_a + G(y - K x_a)$, $G = S_a K^T (K S_a K^T + S_e)^{-1}$	Linear case

Appendix C. Answers and hints

Numerical answers below were computed with the book's own code. Where an answer is a method, it says so.

The three-question assessment from the original project plan

1. **What does a satellite methane instrument typically measure to infer atmospheric methane?** Electromagnetic radiation affected by methane absorption (Chapter 2). It does not photograph molecules, measure pipeline temperature, or use wind alone.
2. **What does detecting a methane plume tell us?** An enhancement was observed under particular measurement conditions (Chapters 3, 9, 10). It does not mean the source emits all year, that annual emissions are known, or that every nearby facility is emitting.
3. **Why are wind measurements important for estimating methane emissions?** They help relate the observed plume to its transport and emission rate (Chapter 11). They do not set the orbit, define molecular structure, or remove uncertainty; wind error is one of the largest terms in the budget.

Chapter 1

1. Radar uses microwaves that pass through cloud and its own illumination; smooth water returns little energy, so it appears dark. Optical sensors need sunlight to reach the ground and reflect back, which cloud blocks.
2. From Level 3 you cannot learn: how noisy individual soundings are; whether a plume exists on a given day; how many times a place was actually observed. (Others are acceptable: quality-flag removal, local-time effects, retrieval artefacts.)
3. NDVI: empirical, validate against ground vegetation. Rainfall: empirical calibration against gauges. XCO_2: physical retrieval, validate against ground-based reference networks and aircraft profiles.

Chapter 2

1. 20 ppm × 100/1013 = **1.97 ppm** (the code gives 1.970).
2. With lines twice as wide the deepest line absorbs 78 % instead of 95 %, so it is less saturated; the +4 ppm change in band-mean radiance becomes −0.099 % instead of −0.073 %. Wider lines cover more of the band and saturate less, so a given change in CO_2 produces more change in the signal.
3. Dry air is the conserved quantity: surface pressure and water vapour change the total air above a point but not the number of dry-air molecules relative to CO_2, so a ratio to dry air does not change with weather.

Chapter 3

1. Spacing at the equator is 172 km; a 1° box is about 111 km wide, so about **0.65 tracks per cycle** (one pass in roughly every cycle and a half on average). A 10.3 km swath covers about **6 %** of the ground.
2. Four reasons: the plant may be emitting at a different time of day; the pass may have been 100 km away (the swath covers 7 % of the box); cloud or poor quality may have removed the soundings near the plant; wind carried the plume off-track; the enhancement was below the detection floor.
3. (a) Sentinel-1 radar; (b) a high-resolution imaging spectrometer or commercial methane imager, tasked at the site; (c) Landsat for the long record, or Sentinel-2 for finer detail from 2015.

Chapter 4

1. Method: add `assert 0.05 < good_fraction < 0.95`. A sudden change in the fraction of good data is the earliest sign that a flag has changed meaning or a file is corrupt.
2. Johannesburg to Pretoria: about **51.8 km** in both UTM and geodesic (51.775 and 51.788 km); the difference is about 13 m.
3. Fields: collection (sentinel-2-l2a), bounding box of the city, datetime range for the month, and `eo:cloud_cover` less than 20.

Chapter 5

1. About **5 %** of soundings have a second source within 40 km, compared with 39 % that have the nearest source within 40 km.
2. Method: sort by time, compute differences, `new = diff > gap_s`, `pass_id = new.cumsum()`.
3. The unflagged bins are the denominator for every rate and frequency; deleting them removes the ability to compute detection frequency and the control comparison.

Chapter 6

1. Method: set `shadow[...]=False`; precision rises because the false water goes away; moving Otsu by ±0.1 changes the area by a few percent.
2. Requiring NIR above 0.05 removes shadow pixels (NIR 0.03) but also removes true water (NIR 0.03). A mask that uses a feature shared by the target removes the target too; choose masks based on a different physical property (for example terrain shadow from a DEM).
3. NDVI is highest over the vegetation patches (about +0.8). After a crop fire NDVI falls sharply there.

Chapter 7

1. Method: vary the drought factor, then compare the monthly z-score and SPI. For skewed rainfall the two diverge most in the dry tail, so the gap appears at milder droughts than you expect.
2. Method: accumulate with `rolling(3).sum()` over the continuous monthly series, then fit gamma per calendar month. Expect the 2015–2016 values to be more negative and more persistent.
3. Method: per pixel and calendar month compute min and max over 20 years; VCI = 100 × (NDVI − min)/(max − min). For missing months, leave missing, or aggregate to seasons with a minimum-count rule.

Chapter 8

1. At ρ = 0.6 the false-alarm rate is **6.6 %** with n = 50 versus 5.8 % with n = 20: it does not improve. The inflation factor tends to √((1+ρ)/(1−ρ)) = 2 whatever n is; more correlated soundings do not buy independence.
2. With offset spread 0.1 the honest error bar is about **2.5×** the naive one (instead of 5×). The problem matters in proportion to how large the shared pass-level error is compared with the bin noise.
3. Method: a 30-number sample with one value of 1000 has mean and standard deviation dominated by it; the median and 1.4826·MAD barely move.

Chapter 9

1. Method: add `artefact = 1.0*exp(-0.5*((x-60)/8)**2)` to `y`; the running median background follows part of it (window 100 km) and the artefact appears as a weak anomaly. Spot it by repeating on other days and by plotting far-field anomalies.

2. With a 0.5 ppm floor the false-alarm rate rises sharply for correlated errors (several percent), while for independent errors it stays small. This shows the floor, not the z-score, was protecting the detection.
3. Method: group events by place (hotspot id) and count distinct dates; require two or more. For a source observed 25 times a year, if a single-pass detection probability is p the chance of two or more is $1 - (1-p)^{25} - 25p(1-p)^{24}$; this cuts false alarms sharply and costs sensitivity.

Chapter 10

1. About **110 passes** are needed (n = 100 gives width 0.155, n = 120 gives 0.142). At about 27 usable passes a year that is **about four years**.
2. Near: within a radius of the facility list; far: at least 2× that distance from any facility. The far zone is not clean if unlisted facilities, wetlands, or landfills lie there, or if artefact-prone surfaces are concentrated in the near or far zone.
3. 4 of 10: 0.17–0.69. 40 of 100: 0.31–0.50. The second is far more informative; the first is compatible with almost any frequency.

Chapter 11

1. Peak enhancement scales as 1/U and the minimum detectable emission as U. At 2 m/s the peak is 1.14 ppm and the minimum detectable emission is about 264 kg/s; at 8 m/s the peak is 0.29 ppm and the minimum is about 1,056 kg/s. Low wind favours detection, but the steady-plume model breaks down at very low speeds.
2. Method: run the filter within each `surf` class. The bias shrinks because each class has a tighter background covariance, and the noise floor typically falls.
3. Average emission is 50 kg/s, about 1.6 Mt/yr. Scaling a single “on” observation (100 kg/s) to a year would give about 3.2 Mt/yr, twice the truth. Improve it with many observations and an intermittency estimate (the fraction of observations with and without emission).

Chapter 12 to 19

These exercises ask for your own design work, so answers are checklists, not numbers. For each: have you stated the question in four parts, named a control, stated what is verified, and chosen an independent validation dataset? Chapter 17, exercise 1: a 5 t/h methane source peaks at about **14 ppb at 10 km, 7 ppb at 20 km and 3.6 ppb at 40 km** in the Chapter 11 model, so against single-pixel noise of order ten ppb or more only the nearest region could stand out, and averaging over days and pixels is required.

Appendix D. Further reading and resources

> **Check before you cite**
> These are pointers from general knowledge, listed by author, year and topic so you can find them. I have not been able to check titles, volume numbers or availability from here. Confirm each reference before quoting it.

Textbooks

- **Remote sensing, general:** Lillesand, Kiefer and Chipman, *Remote Sensing and Image Interpretation*; Jensen, *Introductory Digital Image Processing* and *Remote Sensing of the Environment*; Campbell and Wynne, *Introduction to Remote Sensing*.
- **Atmospheric retrieval and inversion:** Rodgers (2000), *Inverse Methods for Atmospheric Sounding*; the standard reference for averaging kernels, a priori and error analysis.
- **Atmospheric radiation:** Liou, *An Introduction to Atmospheric Radiation*; Petty, *A First Course in Atmospheric Radiation*.
- **Atmospheric transport and inverse modelling for emissions:** Jacob, *Introduction to Atmospheric Chemistry* and the open course notes that accompany it; Brasseur and Jacob, *Modeling of Atmospheric Chemistry*.
- **Statistics:** Wilks, *Statistical Methods in the Atmospheric Sciences*; Efron and Tibshirani, *An Introduction to the Bootstrap*; Cressie and Wikle, *Statistics for Spatio-Temporal Data*.

Papers and reviews worth finding

- Eldering and co-authors (2017) on the OCO-2 mission and first science results; and Crisp and co-authors on the OCO-2 retrieval and its validation.
- Nassar and co-authors (2017, 2021) on quantifying CO_2 emissions from power plants and cities with OCO-2.
- Jacob and co-authors (2022), a review of quantifying methane emissions from global to point-source scales with satellite data.
- Varon and co-authors (2018) on quantifying methane point sources with fine-scale plume imaging, and later work on the IME method and detection limits.
- Lauvaux and co-authors (2022) on global methane ultra-emitters detected with TROPOMI.
- Olofsson and co-authors (2014), *Good practices for estimating area and assessing accuracy of land change*, Remote Sensing of Environment.
- McKee, Doesken and Kleist (1993) on the standardised precipitation index.
- McFeeters (1996) and Xu (2006) on NDWI and MNDWI.

Open data and tools

- NASA Earthdata (GES DISC and others); Copernicus Data Space Ecosystem; USGS EarthExplorer; Google Earth Engine; Microsoft Planetary Computer; STAC specification.
- WRI Global Power Plant Database (used in the project, CC BY 4.0).
- Libraries: `numpy`, `pandas`, `xarray`, `scipy`, `scikit-learn`, `scikit-image`, `pyproj`, `shapely`, `geopandas`, `rasterio`, `matplotlib`, `folium`, `earthaccess`, `sqlalchemy`, `geoalchemy2`, and PostgreSQL with PostGIS.

Mission documentation to read for any dataset you use

For each product: the **algorithm theoretical basis document** (how it is retrieved), the **product user guide** (variables, flags, fill values), the **validation report** (how accurate it is, where), and the **known issues** page. Reading those four documents before using a dataset is the single most valuable habit in this book.

Appendix E. Checklists

Before starting a project

- [ ] Four-part question: quantity, place, period, decision.
- [ ] What would a convincing answer look like? What would convince me it is no?
- [ ] What could fool me?
- [ ] Sensor, product, version, DOI, licence recorded.
- [ ] The five dataset questions answered (Chapter 1).
- [ ] Sampling and the denominator understood (Chapter 3).
- [ ] Independent validation data identified.
- [ ] Synthetic test bench planned, with artefacts and correlated noise.

While building

- [ ] Configuration in one frozen object; parameters written with each result.
- [ ] Tidy table, units in names, keep non-detections.
- [ ] Quality-control report: what each rule removed.
- [ ] Every statistic uses the right unit of replication.
- [ ] Each threshold has a measured false-alarm rate and a sensitivity run.
- [ ] Known-answer, calibration, integration and regression tests.
- [ ] Every figure opened and looked at.
- [ ] Synthetic outputs stamped; never mixed with real.

Before publishing

- [ ] README: purpose, status table, quick start, real-data steps, method, limitations, licences, results.
- [ ] Clean-environment install and demo run verified.
- [ ] Tests pass on at least two library versions.
- [ ] No secrets, no raw data, no synthetic outputs committed.
- [ ] Notebook executed top to bottom, with no errors in its saved outputs.
- [ ] Results section filled only from a real run.
- [ ] Claims sit on the right rung of the ladder.
- [ ] A friendly critic has read the status table.

When a result surprises you

- [ ] Is it too good? Make the simulation harder.
- [ ] Is it the same at other thresholds, radii, dates and with a different background?
- [ ] Does the control region show the same effect?
- [ ] Is there a boring explanation (surface, topography, aerosol, city, shadow)?
- [ ] What would a sceptical expert ask? Answer it in the README.